Database development with PL/SQL INSY 8311

ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Instructor:

- Eric Maniraguha | eric.maniraguha@auca.ac.rw | LinkedIn Profile

6h00 pm – 8h50 pm

- Monday A -G209
- Tuesday B-G209
- Wednesday E-G209
- Thursday F-G308

April 2025

Database development with PL/SQL

Reference reading

- What is Package in Oracle?
- Pipelined Table Functions
- Pipelined Table Functions
- What is PL/SQL? Explained
- SQL CASE Statement with Multiple Conditions
- Database Security
- 7 Coding PL/SQL Procedures and Packages

Lecture 09 – PL/SQL Packages (Oracle) & Pipelined Functions

Introduction: Package vs Data Pipeline

In modern data systems, both **PL/SQL Packages** and **Data Pipelines** play a vital role in handling and processing data effectively—**but at different layers** of the system.

- A **PL/SQL Package** is used **inside the database** to organize and encapsulate business logic, such as procedures, functions, and variables. It improves **modularity, performance, and reusability** of database code.
- A **Data Pipeline**, on the other hand, is used to **move and transform data across systems**—from raw data sources to final destinations like data warehouses or dashboards. It ensures **automation, consistency, and scalability** in data flow.

Together, they support **end-to-end data processing**:
Packages manage logic within the database, while **pipelines** manage how data gets to and from the database in a reliable and structured way.

PL/SQL Package

What is a PL/SQL Package?

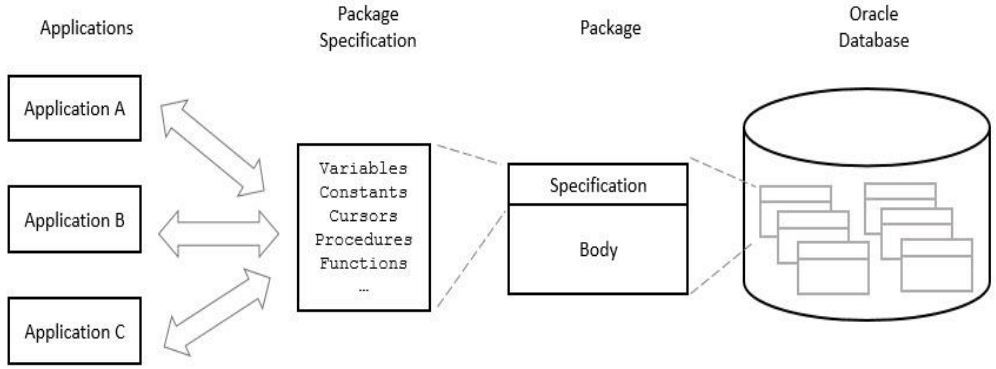
A **PL/SQL Package** is a schema object in Oracle that **groups related procedures, functions, variables, constants, cursors, and exceptions**. It promotes modular programming and is stored in the database.

Package Specification (Required)

- Declares **public objects** (procedures, functions, variables, cursors, etc.).
- These objects are accessible **outside the package**.
- If it declares subprograms or cursors, a body **must** be provided.

Package Body (Optional)

- Implements the subprograms and cursors declared in the specification.
- May also define **private objects**, accessible only within the package.
- Can include:
  - **Initialization section** (runs once per session).
  - **Exception-handling section** (handles internal errors).

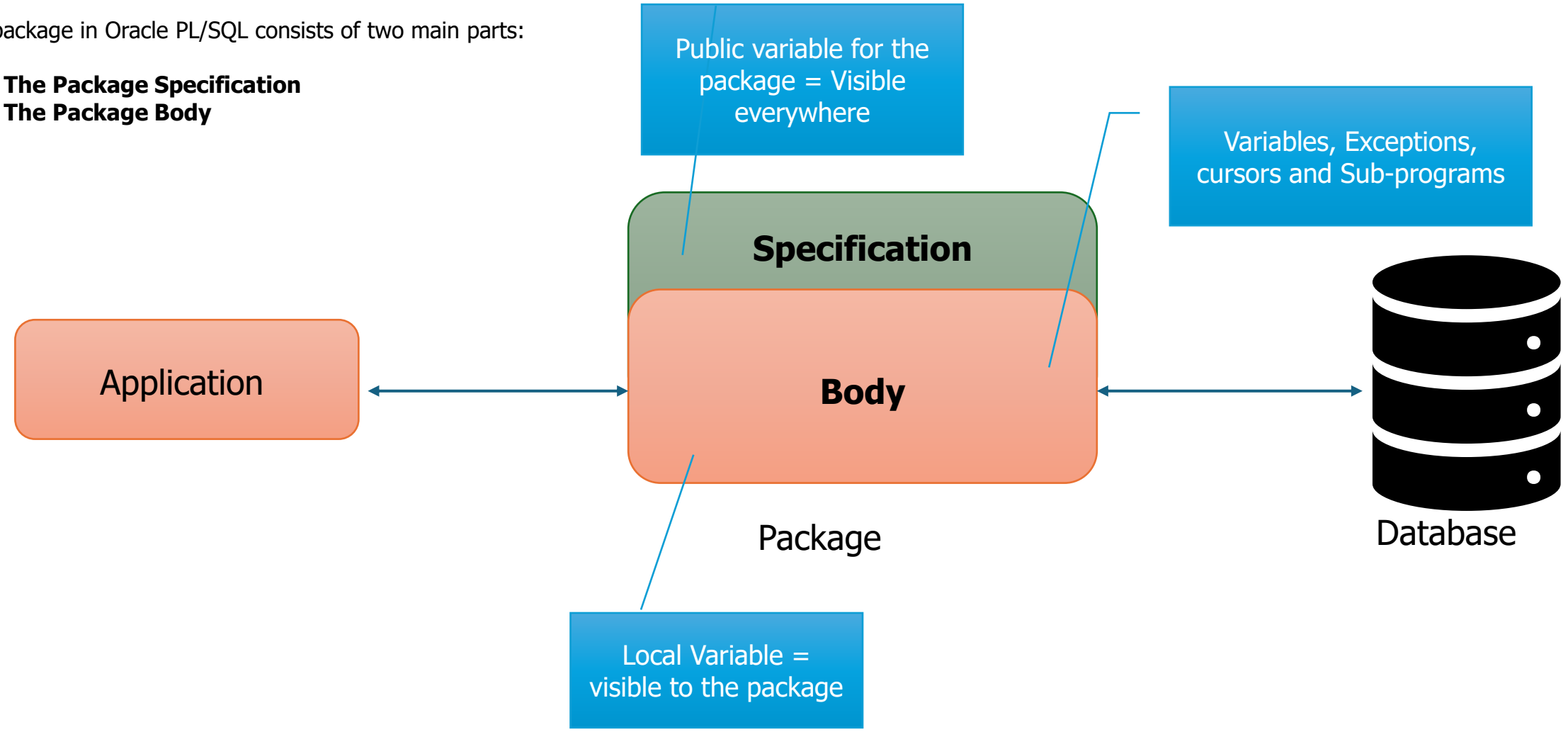


Source Image: https://www.oracletutorial.com/plsql-tutorial/plsql-package/

PL/SQL Package Structure Diagram

A package in Oracle PL/SQL consists of two main parts:

1. **The Package Specification**
2. **The Package Body**



Advantages of PL/SQL Packages

1. Modular Code

- Groups logically related procedures, functions, variables, etc.
- Enhances **code reusability**, **readability**, and **maintainability**.

2. Encapsulation & Implementation Hiding

- Only the **package specification** is exposed to users.
- Internal logic remains hidden in the **package body**.
- Allows changes to the body **without affecting** dependent applications.

3. Improved Performance

- Oracle loads the entire package into memory on the **first reference**.
- Reduces **disk I/O** and improves **execution speed** for subsequent calls.

4. Avoids Unnecessary Recompiling

- Modifying the body **does not require recompiling** dependent objects.
- Only changes to the **specification** trigger recompilation—**minimizing disruption**.

5. Simplified Authorization Management

- Permissions can be granted at the **package level**.
- Makes **security and access control** easier and more efficient.

PL/SQL Package Syntax

1. Package Specification (Header)
Declares public procedures, functions, variables, constants, exceptions, etc.

```
CREATE OR REPLACE PACKAGE package_name AS

   -- Declarations (visible outside the package)
   PROCEDURE procedure_name(parameters);

   FUNCTION function_name(parameters) RETURN datatype;

   -- Public variables, constants, cursors, exceptions (optional)
END package_name;
```

```
CREATE OR REPLACE PACKAGE BODY package_name AS
   -- Procedure implementation
   PROCEDURE procedure_name(parameters) IS
   BEGIN
      -- logic
   END;

   -- Function implementation
   FUNCTION function_name(parameters) RETURN datatype IS
   BEGIN
      -- logic
   END;

   -- Private procedures/functions/variables (optional)
END package_name;
```

2. Package Body (Implementation)
Defines the procedures and functions declared in the specification.
Can also include private logic only used inside the package.

Example I : Greeting Package

Drop the package, created before

```
-- Attempt to drop the PL/SQL package named greeting_pkg
BEGIN
  -- This command removes the entire package (specification and body)
from the database
  EXECUTE IMMEDIATE 'DROP PACKAGE greeting_pkg';
EXCEPTION
  -- Handle the case where the package doesn't exist
  WHEN OTHERS THEN
    -- If the error is not about the object not existing, re-raise it
    IF SQLCODE != -4043 THEN
      RAISE;
    END IF;
    -- Optionally, log or handle the specific error here
END;
```

Package Specification

```
-- Create or replace the package specification for greeting_pkg
CREATE OR REPLACE PACKAGE greeting_pkg AS
  -- Declaration of the procedure say_hello with a parameter name of type VARCHAR2
  PROCEDURE say_hello(name VARCHAR2);

  -- Declaration of the function welcome_message with a parameter name of type VARCHAR2
  -- It returns a VARCHAR2 type result
  FUNCTION welcome_message(name VARCHAR2) RETURN VARCHAR2;
END greeting_pkg;
/
```

Package Body

```
-- Create or replace the package body for greeting_pkg
CREATE OR REPLACE PACKAGE BODY greeting_pkg AS
  -- Definition of the say_hello procedure
  PROCEDURE say_hello(name VARCHAR2) IS
  BEGIN
    -- This procedure outputs a hello message using DBMS_OUTPUT
    DBMS_OUTPUT.PUT_LINE('Hello, ' || name || '!');
  END;

  -- Definition of the welcome_message function
  FUNCTION welcome_message(name VARCHAR2) RETURN VARCHAR2 IS
  BEGIN
    -- This function returns a personalized welcome message
    RETURN 'Welcome, ' || name || '!';
  END;
END greeting_pkg;
/
```

Testing the Greeting Package

```
-- To test the say_hello procedure
BEGIN
   greeting_pkg.say_hello('John');
END;
/
```

```
ello, John!


L/SQL procedure successfully completed.
```

```
-- To test the welcome_message function
DECLARE
   message VARCHAR2(100);
BEGIN
   message := greeting_pkg.welcome_message('John');
   DBMS_OUTPUT.PUT_LINE(message);
END;
/
```

```
Welcome, John!


PL/SQL procedure successfully completed.
```

```
-- Calling the procedure
EXEC greeting_pkg.say_hello('Alice');
```

```
Hello, Alice!


PL/SQL procedure successfully completed.
```

```
-- Calling the function
SELECT greeting_pkg.welcome_message('Alice') FROM dual;
```

	GREETING_PKG.WELCOME_MESSAGE('ALICE')
1	Welcome, Alice!

Example II : Greeting Package

Step 1: Create the Package Specification

```
CREATE OR REPLACE PACKAGE my_package IS
  FUNCTION public_function(p_val NUMBER) RETURN NUMBER;
END my_package;
/
```

Step 2: Create the Package Body

```
CREATE OR REPLACE PACKAGE BODY my_package IS

  -- Private function (not declared in the spec, so not visible outside)
  FUNCTION private_helper(p_val NUMBER) RETURN NUMBER IS
  BEGIN
    RETURN p_val * 2;
  END;

  -- Public function (declared in the spec)
  FUNCTION public_function(p_val NUMBER) RETURN NUMBER IS
  BEGIN
    -- Call the private function
    RETURN private_helper(p_val) + 10;
  END;

END my_package;
/
```

Usage

```
SELECT my_package.public_function(5) FROM dual;
```

But you cannot call private_helper directly from outside the package:

```
-- This will cause an error
SELECT my_package.private_helper(5) FROM dual;
```

Summary

- Functions/procedures declared **only in the package body** are **private to the package**.
- They can be freely used within any other function/procedure inside the same package.
- Only those declared in the **specification** are **public** and accessible outside.

PL/SQL Packages Without Specification - Summary

Is It Allowed?

- Yes, Oracle allows package bodies without specifications.
- Used only for **internal/private procedures or logic**.

Characteristics:

- **No public interface** → contents are **not accessible** externally.
- Cannot be called from SQL or other packages.
- Only useful for **internal calls** (e.g., from triggers, internal procedures).

Limitations:

Aspect	Effect
Visibility	No external access
Reusability	Cannot be reused outside the package
Maintainability	Lacks documentation of intended usage (no spec)
Best Practice	Avoid unless needed for purely internal logic

Use Cases:

- Internal utilities or logging.
- Placeholder for future public spec.
- Logic only triggered by internal DB processes.

🚫 **Not Recommended For:**

- APIs or reusable modules
- Team development and long-term maintenance

Example: Package Body Without Specification

SQL*Plus

```
CREATE OR REPLACE PACKAGE BODY my_hidden_pkg IS

  PROCEDURE log_msg(p_msg VARCHAR2) IS
  BEGIN
    DBMS_OUTPUT.PUT_LINE('Log: ' || p_msg);
  END;

END my_hidden_pkg;
/
```

- This **compiles** successfully.
- But if you try to call *my_hidden_pkg.log_msg('Hello')* from outside, you'll get:

```
ORA-06550: line 1, column 7:
PLS-00201: identifier 'MY_HIDDEN_PKG.LOG_MSG' must be declared
```

Scenario Question: Creating packages and procedures.

Create a PL/SQL package and procedure to manage and display transaction data using an associative array.

Requirements:

1. Package trans_data:
- Define record types for TimeRec (minutes, hours) and TransRec (category, account, amount, time_of).
- Declare a constant minimum_balance (10.00) and an integer number_processed.
- Define an exception insufficient_funds.

2. Package aa_pkg:
- Define an associative array type aa_type indexed by VARCHAR2(15) and containing integers.

3. Procedure print_aa:
- Accept an associative array aa and print each element and its key.

4. Test Block:
- Initialize an associative array with keys 'zero', 'one', and 'two' and values 0, 1, and 2. Call print_aa to display the results.

Expected Output:

0 zero
1 one
2 two

This question tests understanding of:
- Creating packages and procedures.
- Working with records and associative arrays (index-by tables).
- Using FIRST, NEXT, and DBMS_OUTPUT to print data in PL/SQL.

Would you like to adjust the question or add more details?

Solution: Creating packages and procedures.

1. Package trans_data: Defines a package to manage transaction data.

```
CREATE OR REPLACE PACKAGE trans_data AS
  -- Define a record type to hold time information (minutes and hours)
  TYPE TimeRec IS RECORD (
    minutes SMALLINT,
    hours   SMALLINT);

  -- Define a record type for transaction data, including a nested TimeRec
  TYPE TransRec IS RECORD (
    category VARCHAR2(10),
    account  INT,
    amount   REAL,
    time_of  TimeRec);

  -- Constant for the minimum balance required in the transaction
  minimum_balance     CONSTANT REAL := 10.00;

  -- Variable to keep track of the number of transactions processed
  number_processed    INT;

  -- Exception to handle insufficient funds
  insufficient_funds  EXCEPTION;
END trans_data;
/
```

2. **Package aa_pkg**: Defines a collection type (associative array) to store integers indexed by VARCHAR2.

```
CREATE OR REPLACE PACKAGE aa_pkg IS
  -- Define an associative array type (indexed by VARCHAR2(15) and containing integers)
  TYPE aa_type IS TABLE OF INTEGER INDEX BY VARCHAR2(15);
END;
/
```

3. **Procedure print_aa**: This procedure accepts an associative array aa of type aa_pkg.aa_type and prints each element and its corresponding key.

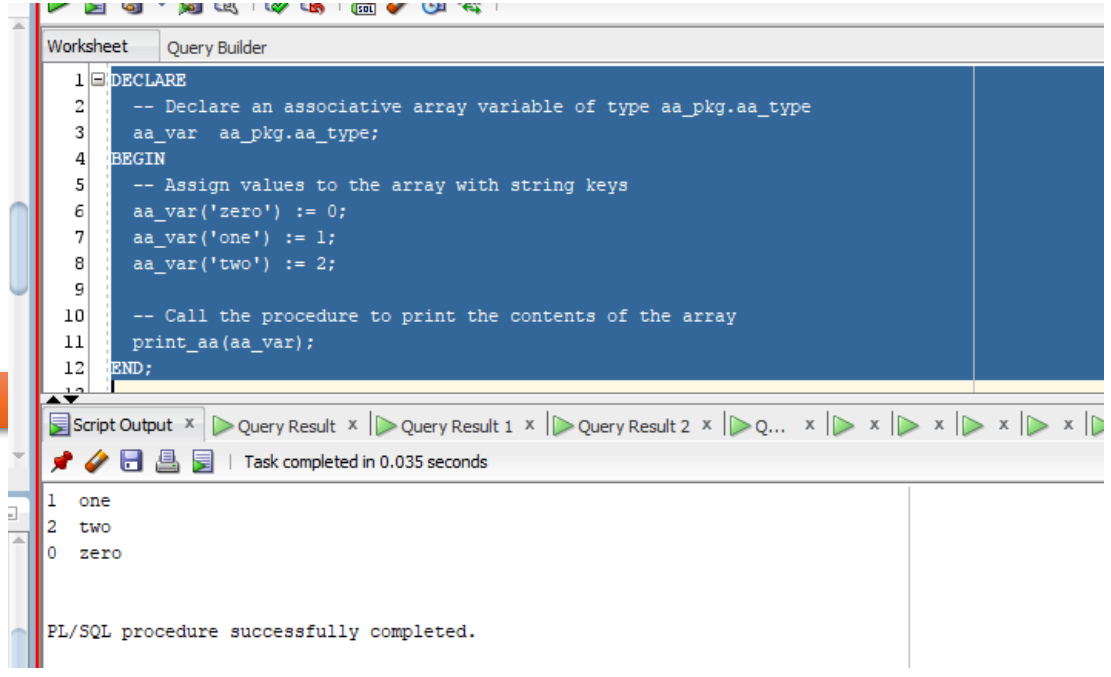
```
CREATE OR REPLACE PROCEDURE print_aa (
  aa aa_pkg.aa_type  -- Accepts the array as a parameter
) IS
  i  VARCHAR2(15);  -- Variable to hold the current index
BEGIN
  -- Start with the first element
  i := aa.FIRST;

  -- Loop through the array until no more elements are left
  WHILE i IS NOT NULL LOOP
    -- Print the value and the index of the array element
    DBMS_OUTPUT.PUT_LINE (aa(i) || ' ' || i);
    -- Move to the next index in the array
    i := aa.NEXT(i);
  END LOOP;
END;
/
```

Solution: Creating packages and procedures.

```
DECLARE
  -- Declare an associative array variable of type aa_pkg.aa_type
  aa_var  aa_pkg.aa_type;
BEGIN
  -- Assign values to the array with string keys
  aa_var('zero') := 0;
  aa_var('one') := 1;
  aa_var('two') := 2;

  -- Call the procedure to print the contents of the array
  print_aa(aa_var);
END;
```

4. **Test Block**: This anonymous block initializes an associative array aa_var, assigns values to it, and then calls the print_aa procedure to display the values.

Output



Scenario Question: Creating a PL/SQL Package for Bonus and Tax Calculations

Question:
You are tasked with creating a PL/SQL package that handles employee bonuses and tax calculations, along with error logging functionality. Follow the steps below:

Part 1: Create the Error Logging Table
1. Create a table called error_log_table with the following columns:
- log_id: A unique identifier (primary key) for each error log entry. Use auto-increment for this column.
- log_time: The timestamp when the error occurred. Set the default value to the current date and time.
- message: A VARCHAR2(4000) column to store the error message.

Write the SQL statement to create the error_log_table.

Part 2: Create the PL/SQL Package
2. Package Specification:
- Define a constant max_bonus with a value of 5000.
- Declare a procedure calculate_bonus that calculates the bonus as 10% of an employee's salary, ensuring the bonus does not exceed max_bonus.
- Declare a function calculate_tax that calculates a 15% tax on a given purchase amount.

3. Package Body:
- Implement the calculate_bonus procedure as described in the specification.
- Implement the calculate_tax function as described in the specification.
- Add a **private procedure log_error** in the package body to handle logging of error messages (leave the logic empty for now).

Part 3: Test the Package
4. Anonymous Block:
- Create an anonymous PL/SQL block that:
  - Calls the calculate_bonus procedure with a salary of 5000 and prints the calculated bonus.
  - Calls the calculate_tax function with a purchase amount of 1000 and prints the calculated tax.

Expected Output:
- Calculated Bonus: 5000
- Calculated Tax: 150

Part 4: Implement Error Logging
5.Modify the log_error procedure to insert error messages into the error_log_table as follows:
- The procedure should accept an error message as a parameter.
- The procedure should insert the SYSDATE (current date and time) along with the message into the table.

Scenario Question: Creating a PL/SQL Package for Bonus and Tax Calculations

```
PROCEDURE log_error(p_message IN VARCHAR2) IS
BEGIN
    INSERT INTO error_log_table (log_time, message)
    VALUES (SYSDATE, p_message);
EXCEPTION
    WHEN OTHERS THEN
        NULL; -- Prevents the logging procedure from raising exceptions
END log_error;
```

Part 5: Test the Error Logging

6.Test the error logging by calling the log_error procedure with a sample message such as Test log entry. After executing the anonymous block, check the error_log_table to confirm that the message was successfully inserted into the table.
Expected Result:

- The table error_log_table should contain an entry with:
  - **Message**: Test log entry
  - **Log Time**: The timestamp of when the log was created.

Additional Instructions:

- Ensure that the error_log_table exists before calling log_error.
- Use SET SERVEROUTPUT ON to display the output of your anonymous block.
- Ensure the user has appropriate privileges to insert into the error_log_table.

Solution: Creating packages and procedures.

Step 1: Create the Error Logging Table

```
CREATE TABLE error_log_table (
    log_id    NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    log_time DATE DEFAULT SYSDATE,
    message   VARCHAR2(4000)
);
```

Step 2: Create the PL/SQL Package Specification

```
-- Package Specification
CREATE OR REPLACE PACKAGE employee_package IS
   -- Declare a constant for the maximum bonus
   max_bonus CONSTANT NUMBER := 5000;

   -- Declare a procedure to calculate bonus
   PROCEDURE calculate_bonus(p_salary IN NUMBER, p_bonus OUT NUMBER);

   -- Declare a function to calculate tax
   FUNCTION calculate_tax(p_purchase_amount IN NUMBER) RETURN NUMBER;

   -- Declare a procedure for error logging (private)
   PROCEDURE log_error(p_message IN VARCHAR2);
END employee_package;
/
```

Step 3: Create the Package Body

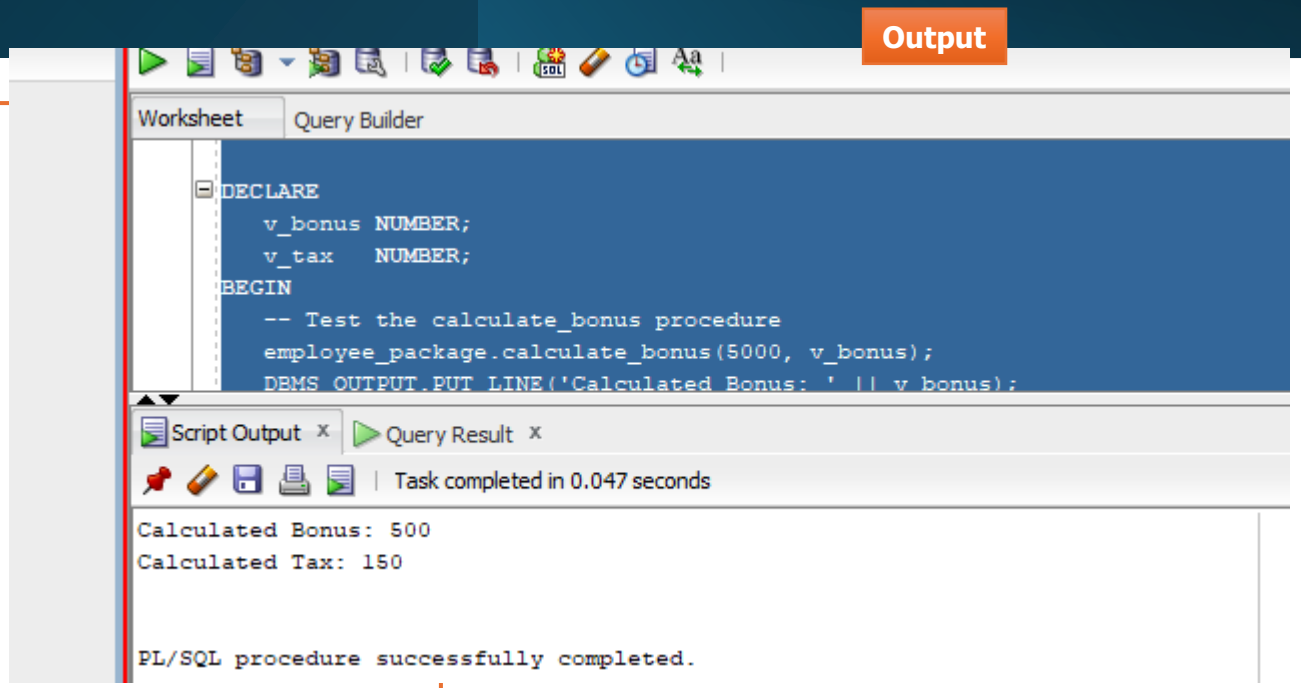
```
-- Package Body
CREATE OR REPLACE PACKAGE BODY employee_package IS

   -- Implementation of the procedure to calculate bonus
   PROCEDURE calculate_bonus(p_salary IN NUMBER, p_bonus OUT NUMBER) IS
   BEGIN
      p_bonus := p_salary * 0.10;  -- Calculate 10% of the salary as bonus
      IF p_bonus > max_bonus THEN
         p_bonus := max_bonus;  -- Cap the bonus at max_bonus
      END IF;
   END calculate_bonus;

   -- Implementation of the function to calculate tax
   FUNCTION calculate_tax(p_purchase_amount IN NUMBER) RETURN NUMBER IS
      v_tax NUMBER;
   BEGIN
      v_tax := p_purchase_amount * 0.15;  -- Calculate 15% tax on the purchase
amount
      RETURN v_tax;
   END calculate_tax;

   -- Private procedure to log errors
   PROCEDURE log_error(p_message IN VARCHAR2) IS
   BEGIN
      INSERT INTO error_log_table (log_time, message)
      VALUES (SYSDATE, p_message);
   EXCEPTION
      WHEN OTHERS THEN
         NULL; -- Avoid infinite loops if logging fails
   END log_error;

END employee_package;
/
```

Solution & Testing: Creating packages and procedures.

Step 4: Test the Package with an Anonymous Block

```
SET SERVEROUTPUT ON;

-- Anonymous Block to test the package

DECLARE
   v_bonus NUMBER;
   v_tax   NUMBER;
BEGIN
   -- Test the calculate_bonus procedure
   employee_package.calculate_bonus(5000, v_bonus);
   DBMS_OUTPUT.PUT_LINE('Calculated Bonus: ' || v_bonus);

   -- Test the calculate_tax function
   v_tax := employee_package.calculate_tax(1000);
   DBMS_OUTPUT.PUT_LINE('Calculated Tax: ' || v_tax);
END;
/
```

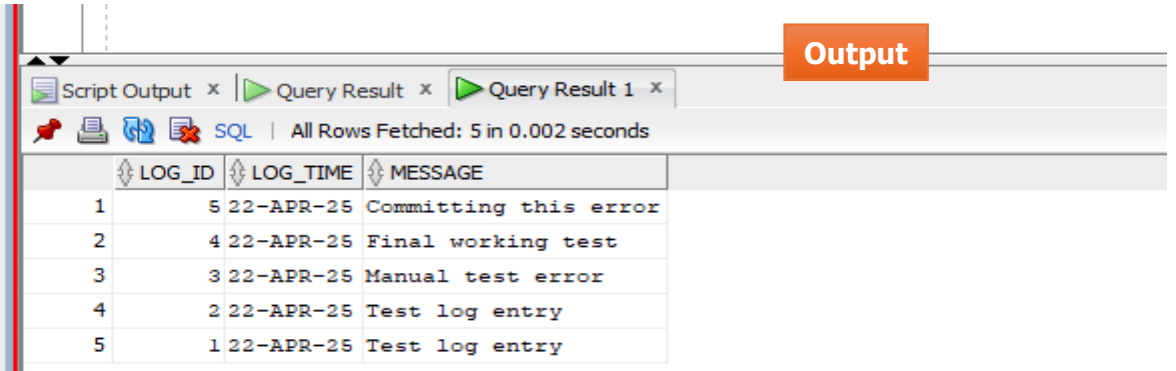
Output



Step 5. Try committing explicitly

```
-- If you're in SQL*Plus, SQL Developer, or another tool that doesn't autocommit:
BEGIN
   employee_package.log_error('Committing this error');
   COMMIT;
END;
/
```

Output



	LOG_ID	LOG_TIME	MESSAGE
1	5	22-APR-25	Committing this error
2	4	22-APR-25	Final working test
3	3	22-APR-25	Manual test error
4	2	22-APR-25	Test log entry
5	1	22-APR-25	Test log entry

PL/SQL Package public (global) and local (private) variables

In PL/SQL, a **package** can contain both **public** (global) and **local** (private) variables. Public variables are accessible to any code that can access the package, whereas local variables are only accessible within the package body.

1. Public (Global) Variables:

- Public variables are declared in the **package specification**.
- They are accessible to any PL/SQL block or program unit that can reference the package, such as procedures, functions, or anonymous blocks.
- These variables act as a global scope for anything that uses the package.

```
CREATE OR REPLACE PACKAGE employee_package IS
   max_bonus CONSTANT NUMBER := 1000; -- public variable

   PROCEDURE calculate_bonus(p_salary IN NUMBER, p_bonus OUT NUMBER);
   FUNCTION calculate_tax(p_purchase_amount IN NUMBER) RETURN NUMBER;
   PROCEDURE log_error(p_message IN VARCHAR2);  -- made public for testing
END employee_package;
/
```

2. Local (Private) Variables:

- Local variables are declared in the **package body**.
- They are only accessible **within the package body** (i.e., by procedures and functions that are defined inside the same package body).
- These variables are used for internal logic or temporary data storage within the package.

```
-- Implementation of the procedure to calculate bonus
   PROCEDURE calculate_bonus(p_salary IN NUMBER, p_bonus OUT NUMBER) IS
   BEGIN
      p_bonus := p_salary * 0.10;  -- Local Variable
      IF p_bonus > max_bonus THEN
         p_bonus := max_bonus;
      END IF;
   END calculate_bonus;
```

Key Differences:

- **Public variables** (like max_bonus) can be accessed outside the package by referencing the package name, whereas **local variables** (like v_temp_bonus) are only visible within the package body.
- Public variables are declared in the **package specification** (IS section), while local variables are declared in the **package body** (BODY section).

PL/SQL Package Global and local variables - Example

Package Specification

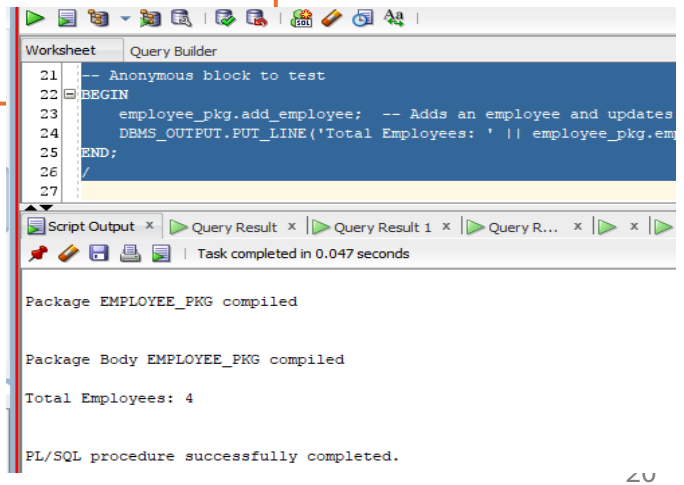
```
-- Package Specification
CREATE OR REPLACE PACKAGE employee_pkg AS
    -- Public variable (accessible outside the package)
    emp_count NUMBER := 0;

    -- Public procedure
    PROCEDURE add_employee;
END employee_pkg;
/
```

Body Package

```
-- Package Body
CREATE OR REPLACE PACKAGE BODY employee_pkg AS
    -- Implementation of the public procedure
    PROCEDURE add_employee IS
    BEGIN
        emp_count := emp_count + 1;
    END;
END employee_pkg;
/
```

Test method

```
-- Anonymous block to test
BEGIN
    employee_pkg.add_employee;  -- Adds an employee and updates emp_count
    DBMS_OUTPUT.PUT_LINE('Total Employees: ' || employee_pkg.emp_count);  -- Accesses
the public variable
END;
/
```



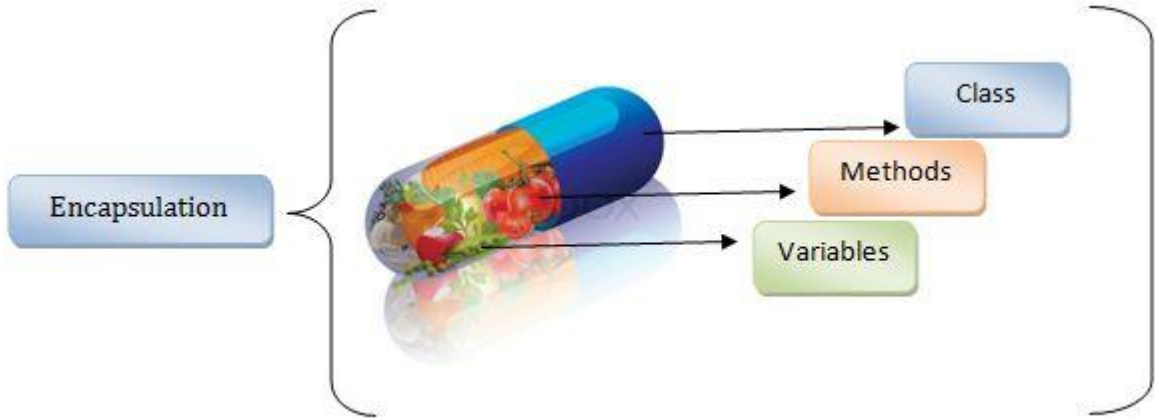
What is Package Encapsulation in PL/SQL?

Encapsulation refers to **hiding the internal implementation details** of a package and **exposing only what's necessary** to the outside world. It is one of the key principles of **modular programming** and helps in:

- Reducing complexity
- Improving maintainability
- Securing internal logic

Structure of a Package
A PL/SQL package has two parts:
1. Package Specification:
1. Declares public elements (procedures, functions, variables, constants).
2. Acts as an interface to users.

2. Package Body:
1. Implements the logic of the procedures and functions.
2. May include private elements (only accessible within the package body).



Source Image: https://logicmojo.com/encapsulation-in-oops

Benefits of Encapsulation in Packages
- **Hides implementation details** from users.
- **Separates interface and implementation**, so you can change the logic without affecting users.
- Prevents misuse of internal logic.
- Allows defining private helper procedures and variables.

PL/SQL Encapsulation example

Example: Encapsulation in Action

```
-- Package Spec (Public Interface)
CREATE OR REPLACE PACKAGE bank_pkg IS
    -- Public Procedure
    PROCEDURE deposit(p_amount NUMBER);
END bank_pkg;
/

-- Package Body (Private Logic Inside)
CREATE OR REPLACE PACKAGE BODY bank_pkg IS
    -- Private variable (not accessible outside)
    balance NUMBER := 0;

    -- Private procedure
    PROCEDURE log_transaction(p_type VARCHAR2, p_amount NUMBER) IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE(p_type || ' of ' || p_amount);
    END;

    -- Public procedure
    PROCEDURE deposit(p_amount NUMBER) IS
    BEGIN
        balance := balance + p_amount;
        log_transaction('Deposit', p_amount);  -- internal call
    END;
END bank_pkg;
/
```

SQL*Plus

Usage

```
BEGIN
    bank_pkg.deposit(1000);  -- Allowed
    -- bank_pkg.balance := 5000;  -- Not allowed: balance is private
    -- bank_pkg.log_transaction('Deposit', 500);  -- Not allowed: log_transaction is private
END;
```

Summary:
Encapsulation via packages allows you to:

- **Expose only what's needed** (via the package spec).
- **Keep internals private** (in the package body).
- Build **secure and maintainable** PL/SQL applications.

Scenario Question: Basic Employee Management System – Encapsulation

A small company wants to automate how it stores and retrieves employee information. The HR team needs a system that allows them to:

1. **Add new employees** with personal and job-related details like name, contact info, hire date, department, role, and salary.
2. **Find the name of a department** an employee belongs to by using the employee's ID.

To make the system organized and reusable, the company wants all related functionality to be grouped in a **PL/SQL package** named *emp_mgmt.*

Student Task

Create a PL/SQL package emp_mgmt with:

- A **procedure** add_employee that inserts a new employee into an employees table.
- A **function** get_department that returns the department name for a given employee ID using the departments table.

Ensure your solution follows good practices like encapsulation and handles missing data gracefully.

Solution: Basic EMS Package Encapsulated

Body Package

```
CREATE OR REPLACE PACKAGE BODY emp_mgmt AS

    -- Implementation of the procedure to add a new employee
    PROCEDURE add_employee(
        p_first_name      VARCHAR2,
        p_last_name       VARCHAR2,
        p_email           VARCHAR2,
        p_phone_number    VARCHAR2,
        p_address         VARCHAR2,
        p_hire_date       DATE,
        p_department_id   NUMBER,
        p_role_id         NUMBER,
        p_salary          NUMBER
    ) IS
    BEGIN
        -- Insert the new employee record into the EMPLOYEES table
        INSERT INTO employees (
            first_name, last_name, email, phone_number, address, hire_date,
            department_id, role_id, salary
        ) VALUES (
            p_first_name, p_last_name, p_email, p_phone_number,
            p_address, p_hire_date, p_department_id, p_role_id, p_salary
        );
    END add_employee;
- Implementation of the function to retrieve the department name
    FUNCTION get_department(p_emp_id NUMBER) RETURN VARCHAR2 IS
        v_department_name VARCHAR2(100);  -- Variable to hold the department
name
    BEGIN
        -- Retrieve the department name based on the employee's departt ID
        SELECT department_name
        INTO v_department_name
        FROM departments
        WHERE department_id = (
            SELECT department_id
            FROM employees
            WHERE employee_id = p_emp_id
        );
        RETURN v_department_name;  -- Return the retrieved department name
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            -- Handle case where no department is found for the given employee
ID
            RETURN NULL;
    END get_department;

END emp_mgmt;
/
```

Using the package

```
/* Using the Package */
BEGIN
    -- Adding a new employee named John Doe with specific details
    emp_mgmt.add_employee(
        p_first_name      => 'John',
        p_last_name       => 'Doe',
        p_email           => 'john.doe@example.com',
        p_phone_number    => '123-456-7890',
        p_address         => '123 Elm Street, Cityville',
        p_hire_date       => TO_DATE('2023-06-15', 'YYYY-MM-DD'),
        p_department_id   => 3,  -- Assuming 3 is the department ID
for IT
        p_role_id         => 5,    -- Assuming 5 is the role ID for
Software Engineer
        p_salary          => 65000 -- Salary amount in the local currency
    );
END;
/
```

Solution: Basic EMS – Package Encapsulation & Result

Testing

```
/*
Retrieving the Department Name for an Employee
To get the department name for a specific employee based on their employee ID,
you can call the get_department function. Below is an example of how to use the function.
*/

DECLARE
    v_dept_name VARCHAR2(100);  -- Variable to hold the department name
BEGIN
    -- Get the department name for employee with ID 1001
    v_dept_name := emp_mgmt.get_department(p_emp_id => 3);

    -- Display the result
    DBMS_OUTPUT.PUT_LINE('Department Name: ' || v_dept_name);
END;
/
```

```
7  DECLARE
8      v_dept_name VARCHAR2(100);  -- Variable to hold the
9  BEGIN
10     -- Get the department name for employee with ID 1001
11     v_dept_name := emp_mgmt.get_department(p_emp_id => 3
12
13     -- Display the result
14     DBMS_OUTPUT.PUT_LINE('Department Name: ' || v_dept_n
15 END;
16 /
17
```

Script Output | Query Result | Query Result 1 | Query R...

Task completed in 0.022 seconds

```
Department Name: Sales


PL/SQL procedure successfully completed.
```

```
1  /*
2  Testing the Code
3  */
4  SELECT * FROM EMPLOYEES WHERE DEPARTMENT_ID = 3;
5
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3 | Query Re...

SQL | Fetched 50 rows in 0.005 seconds

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	ADDRESS	HIRE_DATE	DEPARTMENT_ID	ROLE_ID	SALARY
1	77	John	Niyonzima	johnniyonzima@example.com	0777777848	Kigali, Rwanda	10-SEP-21	3	3	103818
2	1001	Jane	Smith	jane.smith@example.com	5551234567	456 Elm St	11-NOV-24	3	2	59895
3	402	Emp8149	User8677	emp8600user1564@example.com	07300791294	City 3, Rwanda	24-NOV-23	3	2	37886

Testing

```
/*
Testing the Code
*/
SELECT * FROM EMPLOYEES WHERE DEPARTMENT_ID = 3;
```

Explore Advanced Concepts in Package Encapsulation

For better exploration and enhancement of package encapsulation, I highly recommend referring to the advanced exercise available in my GitHub repository.

- **GitHub Link:** PL/SQL Financial Package
- This exercise will help you explore advanced PL/SQL concepts and improve your understanding of package encapsulation, which is a crucial element of clean and maintainable code.

"When they go low, we go higher"
by Michelle Obama

AUTHID DEFINER vs AUTHID CURRENT_USER in PL/SQL Packages

When to Use AUTHID DEFINER vs AUTHID CURRENT_USER

Use Case	Recommended AUTHID
Centralized and secure access to sensitive data	*AUTHID DEFINER (default)*
Modular or reusable code that should respect the caller's privileges	*AUTHID CURRENT_USER*
Multi-tenant applications (users access only their own data)	*AUTHID CURRENT_USER*
Security-critical operations requiring tight control over data access	*AUTHID DEFINER*
You want to hide internal implementation or enforce strict logic	*AUTHID DEFINER*
You want the same code to behave differently based on the caller's privileges	*AUTHID CURRENT_USER*

AUTHID DEFINER vs AUTHID CURRENT_USER in PL/SQL

Aspect	AUTHID DEFINER (Default)	AUTHID CURRENT_USER (Invoker Rights)
Execution Context	Runs with privileges of **owner (definer)**	Runs with privileges of **calling user (invoker)**
Default Behavior	Yes	No (must be explicitly specified)
Caller Needs Direct Access	No	Yes
Use Case	▪ Centralized, secure access to data ▪ Encapsulate complex logic	▪ Reusable, modular code ▪ Multi-tenant or user-specific access
Privilege Check Timing	At **compile time**	At **runtime**
Security Risk / Consideration	Potential **privilege escalation** if not properly controlled	All users must have **direct privileges** on accessed objects
Advantages	- Easy to encapsulate logic - Hide underlying data - Simplifies privilege management	- Promotes modularity - Supports role-based access - Enhances security in multi-user environments

PL/SQL Package Syntax with AUTHID Clause

Basic Syntax

```
CREATE [OR REPLACE] PACKAGE package_name
[AUTHID {DEFINER | CURRENT_USER}]
{IS | AS}

 -- Package specifications
END [package_name];
```

1. Using AUTHID DEFINER (Default)

- **Privileges used**: Of the *owner (definer)*
- **Caller only needs EXECUTE privilege** on the package Good for **centralized, controlled access** to data

```
CREATE OR REPLACE PACKAGE my_pkg
AUTHID DEFINER  -- Optional, this is the default
AS
   PROCEDURE my_proc;
END my_pkg;
/
CREATE OR REPLACE PACKAGE BODY my_pkg
AS
   PROCEDURE my_proc IS
   BEGIN
      -- Code runs with privileges of the package owner
      NULL;
   END;
END my_pkg;
/
```

2. Using AUTHID CURRENT_USER (Invoker Rights)

```
CREATE OR REPLACE PACKAGE my_pkg
AUTHID CURRENT_USER
AS
   PROCEDURE my_proc;
END my_pkg;
/
CREATE OR REPLACE PACKAGE BODY my_pkg
AS
   PROCEDURE my_proc IS
   BEGIN
      -- Code runs with privileges of the caller (invoker)
      NULL;
   END;
END my_pkg;
/
```

- **Privileges used**: Of the *calling user*
- **Caller must have direct privileges** on the underlying objects (like tables)
- Good for **multi-tenant, modular, or reusable code** where each user's permissions matter

Scenario : Managing User Privileges in Oracle PL/SQL

Understanding AUTHID with Users A, B, and C

User	Role
User A	Owner of the PL/SQL package/procedure
User B	Invoker/executor of the procedure
User C	Has data objects (tables) to be accessed

Scenario 1: Using AUTHID DEFINER (Default)

Setup:

- User A creates a procedure read_data that accesses C.employee_data.
- The procedure is created **with AUTHID DEFINER (or default)**.
- User B is granted **EXECUTE** on the procedure.

Flow:

1. User A has been granted **SELECT** on C.employee_data.
2. User A creates procedure read_data using this table.
3. User B executes the procedure.

Result: **Success** - because the code runs with **User A's privileges**, even though User B doesn't have access to C.employee_data.

Note: This is useful for hiding data or providing controlled access via APIs.

Scenario 2: Using AUTHID CURRENT_USER

Setup:

- Same procedure, but this time created with AUTHID CURRENT_USER.
- User B is still only granted **EXECUTE** on the procedure, not direct access to the table.

Flow:

1. User B calls the read_data procedure.
2. Since it's AUTHID CURRENT_USER, the procedure runs with **User B's privileges**.

Result: **Failure** - User B does **not have SELECT privilege** on C.employee_data.
Error: ORA-00942: table or view does not exist (from User B's perspective)
Fix: Grant SELECT on C.employee_data directly to **User B** or to a **role** assigned to B.

Scenario : Managing User Privileges in Oracle PL/SQL – Summary Table

Scenario	Privilege Context	Who Needs Access to C.employee_data	Expected Result
DEFINER Rights	User A (Definer)	Only **User A**	Success
CURRENT_USER Rights	User B (Invoker)	Must grant to **User B**	Fails unless granted

Scenario: Centralized Reporting Package in a Bank

Players:

- **DBA team / IT Department** (User: *central_admin*) — owns a package called report_pkg.
- **Bank branches** (Users*: branch_kigali, branch_musanze*, etc.) — each has a schema with local data, including a table transactions.

Problem:
The central IT department wants to write **one package** to generate reports for any branch's transaction data. But **each branch stores their own transactions table** in their own schema.

If the package is defined with **definer's rights** (default), it will always look in *central_admin.transactions* — which doesn't make sense.

Solution: Use Invoker's Rights
Create the package with AUTHID CURRENT_USER, so that when any branch calls the report procedure, it accesses **that branch's own transactions table**.

Scenario: Centralized Reporting Package in a Bank - uses AUTHID DEFINER (default behavior) |uses AUTHID CURRENT_USER (invoker's rights).

The **central admin** user (central_admin) is responsible for maintaining shared PL/SQL packages.

```
--Each bank branch (branch_kigali, branch_musanze, etc.) has its own transactions table with this structure:

CREATE TABLE transactions (
  trans_id    NUMBER,
  amount      NUMBER,
  trans_date DATE
);
```

1. report_definer_pkg — Definer's Rights (AUTHID DEFINER)

```
-- Created by central_admin
CREATE OR REPLACE PACKAGE report_definer_pkg IS
  PROCEDURE summary_report;
END;
/

CREATE OR REPLACE PACKAGE BODY report_definer_pkg IS
  PROCEDURE summary_report IS
    v_total NUMBER;
  BEGIN
    SELECT SUM(amount) INTO v_total FROM transactions
    WHERE trans_date = TRUNC(SYSDATE);

    DBMS_OUTPUT.PUT_LINE('Definer\'s Rights Report - Total: ' || v_total);
  END;
END;
/
```

This package accesses transactions in the **central_admin schema**, no matter who runs it.

2. report_invoker_pkg — Invoker's Rights (AUTHID CURRENT_USER)

```
-- Created by central_admin
CREATE OR REPLACE PACKAGE report_invoker_pkg AUTHID CURRENT_USER IS
  PROCEDURE summary_report;
END;
/

CREATE OR REPLACE PACKAGE BODY report_invoker_pkg IS
  PROCEDURE summary_report IS
    v_total NUMBER;
  BEGIN
    SELECT SUM(amount) INTO v_total FROM transactions
    WHERE trans_date = TRUNC(SYSDATE);

    DBMS_OUTPUT.PUT_LINE('Invoker\'s Rights Report - Total: ' || v_total);
  END;
END;
/
```

This package accesses transactions in the **caller's schema** — e.g., branch_kigali.transactions if the caller is branch_kigali.

Scenario: Centralized Reporting Package in a Bank – Test Behavior

Assume both packages are granted **EXECUTE** to each branch:

```
GRANT EXECUTE ON report_definer_pkg TO branch_kigali;
GRANT EXECUTE ON report_invoker_pkg TO branch_kigali;
```

From branch_kigali, run:

```
EXEC report_definer_pkg.summary_report;
EXEC report_invoker_pkg.summary_report;
```

Expected Output:

- report_definer_pkg.summary_report
  → Fails (or returns NULL) if central_admin.transactions doesn't exist or is empty
  → Accesses central schema (AUTHID DEFINER)

- report_invoker_pkg.summary_report
  → Works and fetches data from branch_kigali.transactions
  → Accesses caller's schema (AUTHID CURRENT_USER)

Summary Table:

Package	Rights Type	Accesses Table In Schema
report_definer_pkg	AUTHID DEFINER	central_admin
report_invoker_pkg	AUTHID CURRENT_USER	**Calling user**, e.g., branch_kigali

Scenario Question: Implementing Role-Based Access Control in a Banking Application using AUTHID

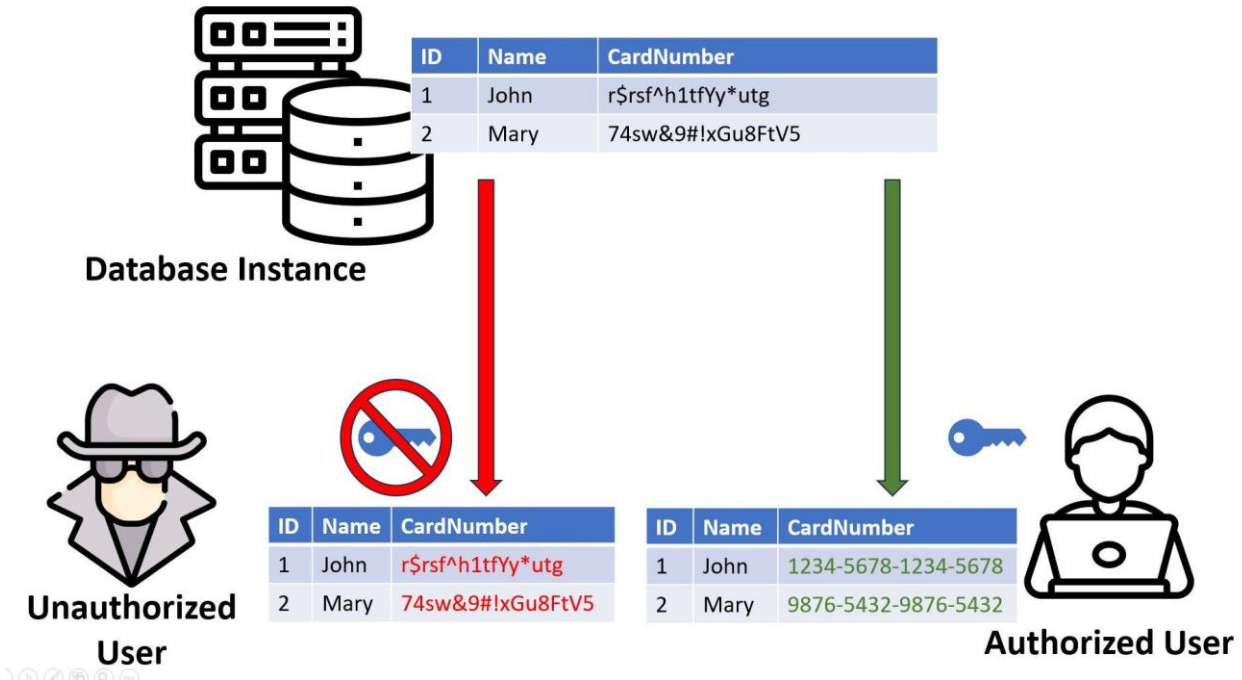
You've presented an interesting banking scenario that leverages PL/SQL's AUTHID clause to implement role-based access control. Let me formulate this as a question and provide a detailed answer.

Question
How can we use PL/SQL's AUTHID clause to implement role-based security in a banking application where different roles (Tellers, Managers, and Auditors) need different levels of access to customer data and transactions?

Answer
Understanding Role-Based Requirements
In a banking application, we need to enforce different privileges for:

- **Tellers**: Create regular transactions
- **Managers**: Approve high-value transactions
- **Auditors**: View but not modify any customer data

Explore my proposed solution in this GitHub repository: https://github.com/ericmaniraguha/PLSQL_AuthID_banque_system. Feel free to review and optimize the implementation.



Scenario Question: Healthcare System with Multi-Level Access

In a healthcare system where users have different roles—such as Doctors, Nurses, and Administrators—how can PL/SQL's AUTHID clause be used to enforce role-based access control to ensure that each user only accesses or modifies patient data according to their privileges?

Scenario Description

In a healthcare information system, various users interact with patient data, but their access levels differ:

- **Doctors**: Can **view and update** sensitive diagnosis and treatment data.
- **Nurses**: Can **update only basic patient information**, such as vitals or contact details.
- **Administrators**: Can **view all patient data**, including medical records, but **cannot modify** any diagnosis or treatment information.

Scenario Question: Multi-Tenant SaaS Application

In a Software-as-a-Service (SaaS) application that serves multiple organizations (tenants), each organization's data must be isolated and accessible only by users from that organization.

- **Problem**: Procedures that access data should be restricted to data that belongs only to the user's organization. For example, get_employee_records should only return employees from the user's organization, not other organizations.

- **Solution with AUTHID**:
  - Using AUTHID CURRENT_USER in procedures that access sensitive or tenant-specific data allows the code to dynamically adapt to the privileges of each user or organization.
  - The database can enforce additional security checks based on the caller's privileges, making sure users can only access their own organization's data.
  - Procedures with broader access, such as global configurations, could still use AUTHID DEFINER to ensure they run with a higher level of privilege.

By combining AUTHID DEFINER and AUTHID CURRENT_USER, the SaaS application can isolate tenant data while still allowing flexible access control.

In each of these scenarios, the AUTHID clause allows developers to control how privileges are applied to stored procedures and packages, ensuring secure and context-specific access in multi-user and multi-role environments.

Implementing Secure Role-Based Access in Oracle PL/SQL Using the AUTHID Clause

In this example:

- If my_package is created with AUTHID DEFINER, the user who owns the package must have privileges to insert into my_table.
- Even if some_user (who does not have insert privileges on my_table) calls insert_value, it will succeed because the execution uses the privileges of the package owner.

Summary

- **AUTHID DEFINER** allows a PL/SQL package or procedure to execute with the privileges of the user who defined it, regardless of who is calling it. This can enhance security and simplify permission management in multi-user databases.
- Conversely, **AUTHID CURRENT_USER** executes with the privileges of the user calling the procedure, which is more flexible but may require careful management of permissions.

Conclusion

The AUTHID clause is a valuable tool for implementing robust security measures in Oracle databases. By understanding its different settings and how to use them effectively, you can ensure that your data is protected and accessed only by authorized users. Remember to choose the appropriate AUTHID setting based on your specific requirements and security considerations.

```
-- Create a table for demonstration
CREATE TABLE my_table (
    id NUMBER PRIMARY KEY,
    value VARCHAR2(100)
);

-- Grant SELECT privilege to a specific user
GRANT SELECT ON my_table TO some_user;

-- Create a package with AUTHID DEFINER
CREATE OR REPLACE PACKAGE my_package AUTHID DEFINER IS
    PROCEDURE insert_value(p_id NUMBER, p_value VARCHAR2);
END my_package;
/

-- Create the package body
CREATE OR REPLACE PACKAGE BODY my_package AS
    PROCEDURE insert_value(p_id NUMBER, p_value VARCHAR2) IS
    BEGIN
        -- The procedure inserts a value into the table
        INSERT INTO my_table (id, value) VALUES (p_id, p_value);
    END insert_value;
END my_package;
/

-- Assume 'some_user' calls the procedure
BEGIN
    my_package.insert_value(1, 'Hello, World!');
END;
/
```

PL/SQL Pattern Package Tutorial

Step 1: Create the Package Specification
This defines the public interface - what functions and procedures are available for use.

```
CREATE OR REPLACE PACKAGE pattern_package AS
    -- Procedure to draw a normal triangle pattern
    PROCEDURE draw_triangle(p_rows IN NUMBER);

    -- Procedure to draw an inverted triangle pattern
    PROCEDURE draw_inverted_triangle(p_rows IN NUMBER);
END pattern_package;
/
```

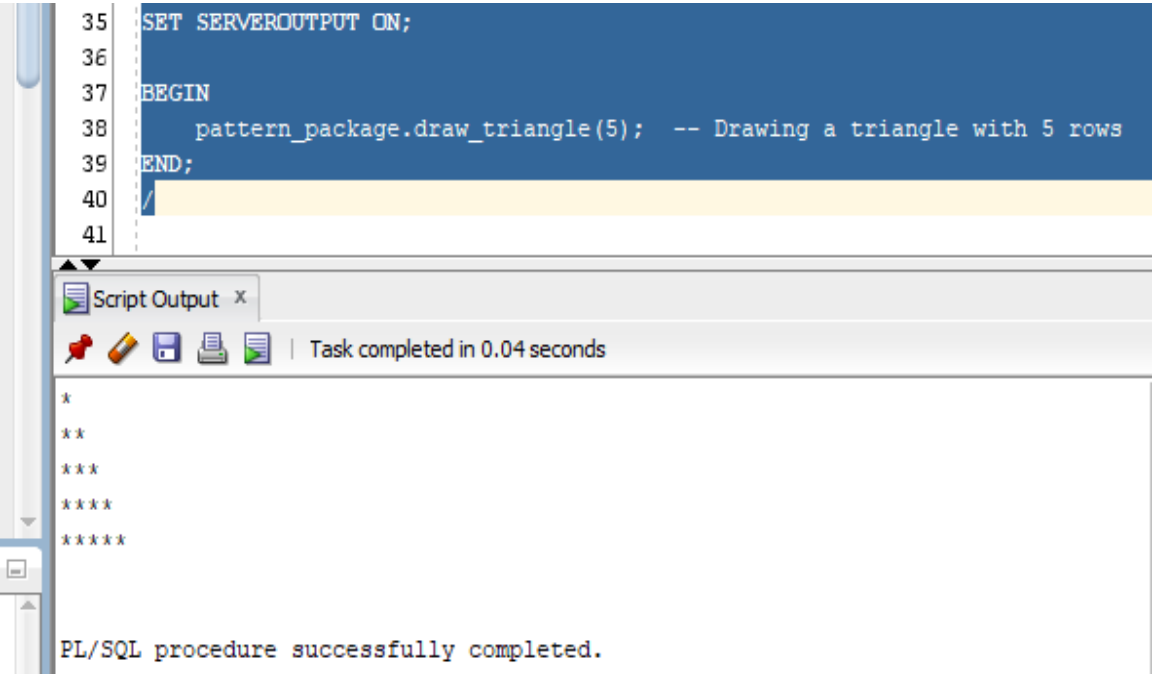
Step 2: Create the Package Body
This implements the actual logic of the package procedures.

```
CREATE OR REPLACE PACKAGE BODY pattern_package AS
    -- Procedure to draw a normal triangle pattern
    PROCEDURE draw_triangle(p_rows IN NUMBER) IS
    BEGIN
        FOR i IN 1..p_rows LOOP
            FOR j IN 1..i LOOP
                DBMS_OUTPUT.PUT('*');
            END LOOP;
            DBMS_OUTPUT.PUT_LINE('');
        END LOOP;
    END draw_triangle;

    -- Procedure to draw an inverted triangle pattern
    PROCEDURE draw_inverted_triangle(p_rows IN NUMBER) IS
    BEGIN
        FOR i IN REVERSE 1..p_rows LOOP
            FOR j IN 1..i LOOP
                DBMS_OUTPUT.PUT('*');
            END LOOP;
            DBMS_OUTPUT.PUT_LINE('');
        END LOOP;
    END draw_inverted_triangle;
END pattern_package;
/
```

Step 3: Calling the Procedures from the Package

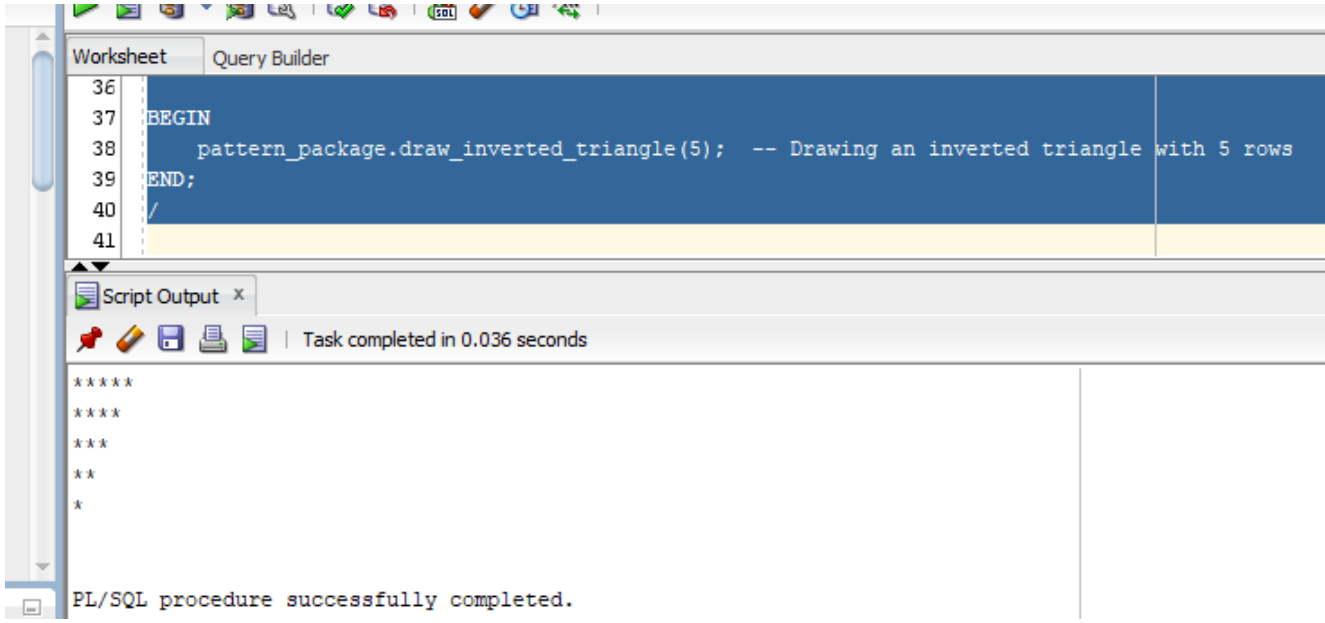
Example 1: Draw a Normal Triangle

```
SET SERVEROUTPUT ON;

BEGIN
    pattern_package.draw_triangle(5);  -- Drawing a triangle with 5
rows
END;
/
```



Example 2: Draw an Inverted Triangle

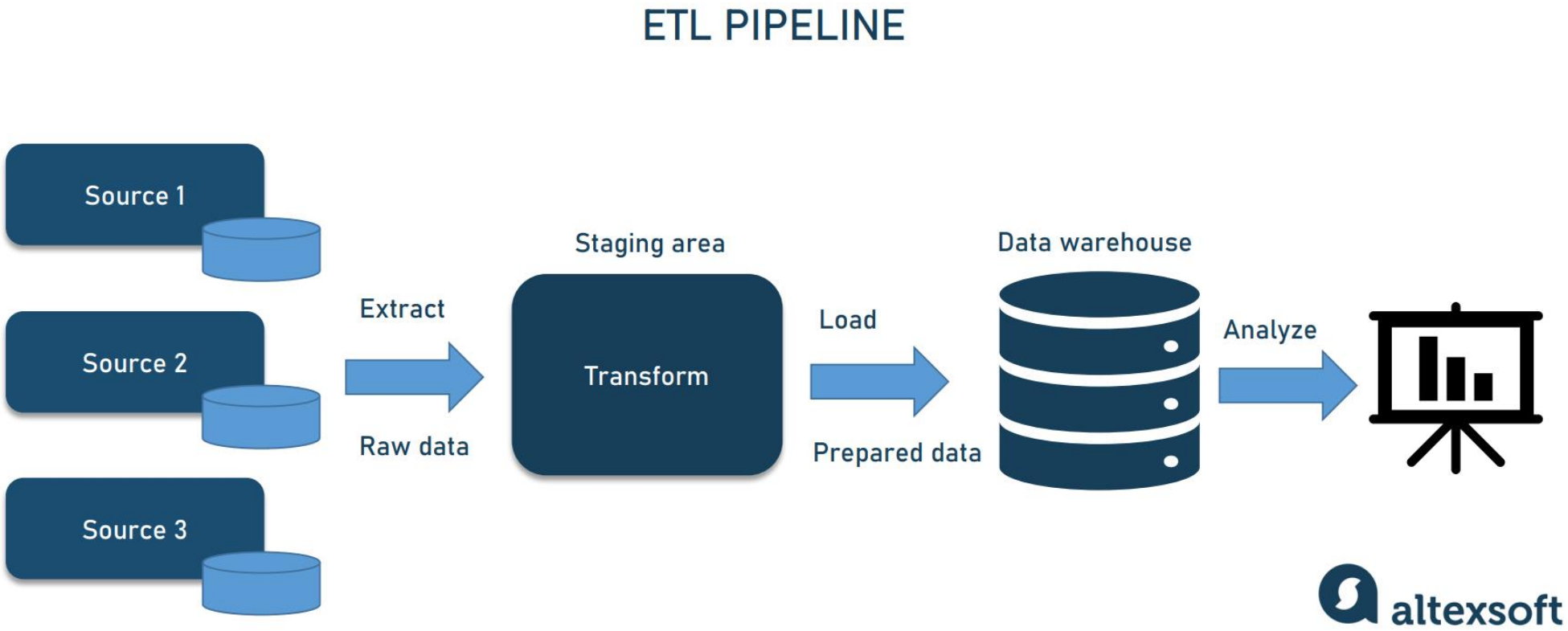
```
SET SERVEROUTPUT ON;

BEGIN
    pattern_package.draw_inverted_triangle(5);  -- Drawing an
inverted triangle with 5 rows
END;
/
```



What is Data Pipeline?

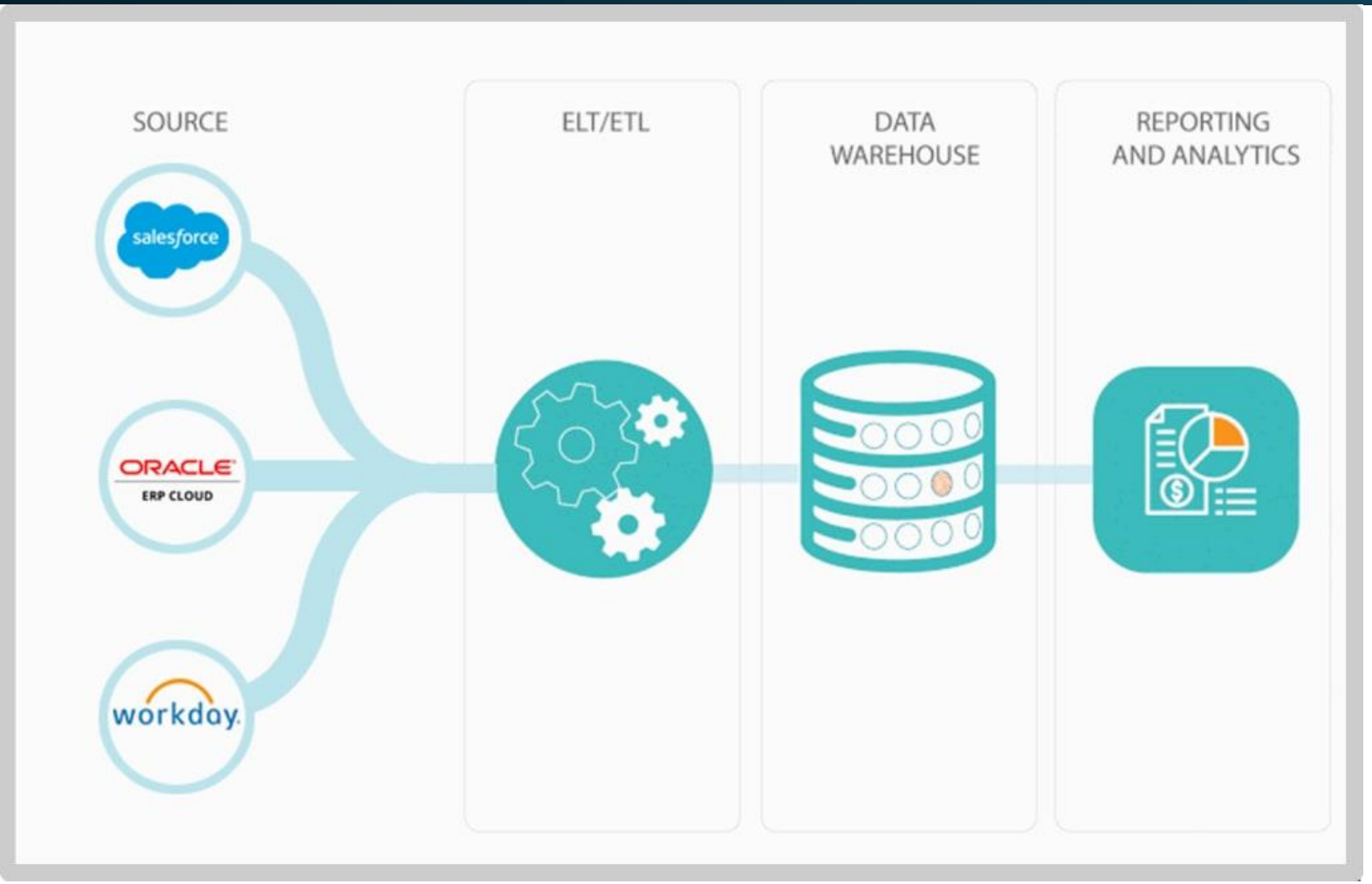
A **data pipeline** is a set of processes or tools that move data from one system (like a database, file system, or API) to another, often through stages of **collection**, **transformation**, and **storage**. Think of it like a factory line for data: raw data comes in, gets cleaned and reshaped, and ends up in a form that's ready for analysis or use in applications.

SQL*Plus



Source Image: https://www.altexsoft.com/blog/data-pipeline-components-and-types/

Data Pipeline Overview



1. Source Systems

- Examples: **Salesforce**, **Oracle ERP Cloud**, **Workday**
- These systems generate and store raw business data.
- In an Oracle environment, data from these sources can be accessed via:
  - **External Tables**
  - **SQL*Loader**
  - **Connectors or APIs**

Source Image: https://vocal.media/01/oracle-cloud-pipelines

Data Pipeline Overview with Oracle Pipelined Table Functions

2. ETL/ELT Process of Data Pipeline, means:

Extract:

- Data is read from the source systems.
- External tables or connectors fetch the raw data.

Transform:

- Transformation logic (e.g., filtering, cleansing, calculations) is applied on-the-fly using **pipelined table functions**.
- No need to write data to intermediate staging tables.

Load:

- Transformed data is immediately inserted into the target data warehouse.
- This reduces latency and improves processing speed.

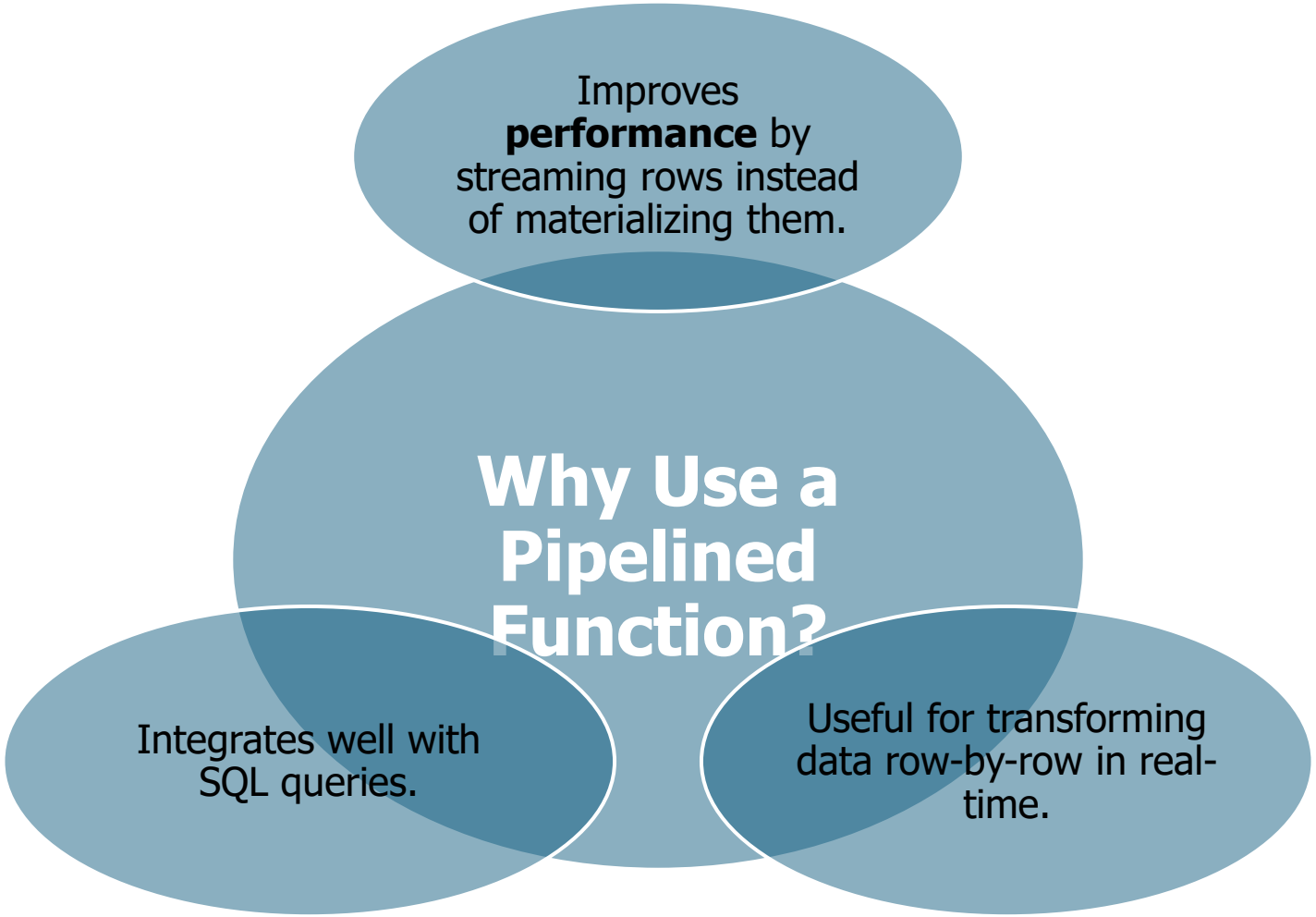
3. Data Warehouse

- Acts as the centralized storage for transformed, ready-to-use data.
- Since pipelined functions stream the data directly, loading is faster and more scalable—ideal for **real-time** or **high-volume** data.

4. Reporting and Analytics

- Final stage where processed data is consumed for:
  - **Business Intelligence**
  - **Dashboards**
  - **Advanced Analytics**
- Because the data can be processed in near real-time, users get timely insights for decision-making.

Why Use a Pipelined Function?



SQL*Plus

where NOT to use Oracle pipelined functions in PL/SQL

Where NOT to Use	Reason	Better Alternative
In **DML operations** (INSERT, UPDATE)	Pipelined functions are **read-only**, not meant to modify data.	Use standard PL/SQL procedures or triggers.
Inside **scalar context**	They return collections, not single values.	Use scalar functions instead.
When **side effects** are involved	Oracle may call functions multiple times in a query, causing unintended behavior.	Use pure SQL or stateless functions.
With **non-deterministic logic**	Cannot be marked DETERMINISTIC, so can't be indexed or cached reliably.	Use standard deterministic functions.
For **small, simple datasets**	Adds unnecessary complexity and overhead.	Use regular table functions or direct SQL.
When using **complex nested data**	Harder to debug and maintain in complex nested structures (e.g., XML/JSON).	Use SQL/XML, JSON_TABLE, or PL/SQL collections.
Without **parallel enable settings**	Pipelined functions won't parallelize unless properly configured.	Use native parallel SQL or properly configured pipelines.
In **function-based indexes**	Not supported due to non-determinism.	Use simple scalar functions instead.

Regular vs Pipelined Table Function in Oracle PL/SQL

Aspect	Regular Table Function	Pipelined Table Function
Definition	Returns a collection as a whole.	Returns rows **one at a time** (streaming fashion).
Return Type	Collection (e.g., TABLE OF RECORD or TABLE OF OBJECT).	Same – but uses PIPELINED keyword.
Keyword	No special keyword.	Uses PIPELINED and PIPE ROW.
Execution Timing	**Waits** until function finishes processing all data before returning.	**Streams** data as it's processed — **faster first row response**.
Performance on Large Data	Slower for large data (entire dataset must be built in memory).	Better — returns data incrementally, reducing memory use.
Parallel Execution Support	Not parallel-enabled.	Can be parallel-enabled with PARALLEL_ENABLE.
Complex Logic Handling	Easier to handle complex transformations at once.	Best for row-by-row lightweight logic.
Memory Usage	High — entire result set stored before returning.	Low — row-by-row output, no full dataset needed in memory.
Use Case	Small datasets, batch processing.	Streaming large datasets, ETL, integration with external sources.
Syntax Example	RETURN my_type;	RETURN my_type PIPELINED; and use PIPE ROW (record);

Key Stages & Features of Good Data Pipeline

Key Stages in a Data Pipeline:

1. **Ingestion** – collecting data from various sources (e.g., databases, APIs, logs).
2. **Processing/Transformation** – cleaning, enriching, normalizing, filtering, or aggregating data.
3. **Storage** – saving the transformed data into a data warehouse, data lake, or database.
4. **Orchestration** – scheduling and managing the workflow of these processes.
5. **Monitoring & Logging** – keeping track of performance, errors, and metrics.

Features of a Good Data Pipeline

Feature	Description
Automation	Can run without manual intervention using schedulers or event triggers.
Scalability	Handles increasing data volume, variety, and velocity.
Reliability	Ensures data gets delivered even if part of the system fails (e.g., retry logic).
Modularity	Built with reusable and independent components (easy to update or replace).
Real-time or Batch	Can be designed for real-time (streaming) or periodic (batch) processing.
Monitoring & Alerts	Provides dashboards or alerts to detect issues in processing.
Data Quality Checks	Validates data formats, values, and completeness.
Security	Implements access controls, encryption, and audit logging.
Logging & Auditing	Keeps track of what was processed and when (useful for debugging).
Version Control & Reproducibility	Enables versioning of data and transformation logic for consistent outputs.

Oracle Pipelined Table Function – Syntax Overview

Step 1: Create the Object Type

```
--Represents the structure of a single row in the result set.
CREATE OR REPLACE TYPE object_type_name AS OBJECT (
    column1_name datatype1,
    column2_name datatype2,
    ...
);
/
```

Step 2: Create the Collection Type or Nested Table Type

```
-- Defines a table of the object type, used as the return type for the pipelined function
CREATE OR REPLACE TYPE collection_type_name AS TABLE OF object_type_name;
/
```

Step 3: Create the Pipelined Table Function

```
CREATE OR REPLACE FUNCTION function_name (
    parameter1 datatype,
    parameter2 datatype
)
RETURN collection_type_name PIPELINED
IS
BEGIN
    FOR record IN (
        SELECT ... FROM ... WHERE ...
    ) LOOP
        PIPE ROW(object_type_name(record.column1, record.column2, ...));
    END LOOP;

    RETURN; -- End of function
END function_name;
/
```

Scenario: Oracle Pipelined Table Functions (1/2)

What is a Pipelined Function?
A **pipelined function** is a **table function** that **returns rows one at a time** as they're produced, instead of waiting until all processing is complete.

This is super useful when:

- You want to **transform or filter data row-by-row**.
- You want to **return results to SQL immediately**, without waiting for the full set.
- You're dealing with **large data** and want better performance via streaming.

Use cases:
A branch wants to generate **daily transactions** as a **table** that can be used in SELECT statements, BI tools, joins, or views.

Step-by-Step: Pipelined Function Example

Step 1: Create a Table Type and Object Type

```
-- Object type representing a row
CREATE OR REPLACE TYPE transaction_rec AS OBJECT (
  trans_id    NUMBER,
  amount      NUMBER,
  trans_date DATE
);
/
-- Table type of those rows
CREATE OR REPLACE TYPE transaction_tab AS TABLE OF transaction_rec;
/
```

Step 2: Create the Pipelined Function

```
CREATE OR REPLACE FUNCTION get_today_transactions
  RETURN transaction_tab
  PIPELINED
IS
BEGIN
  FOR rec IN (
    SELECT trans_id, amount, trans_date
    FROM transactions
    WHERE trans_date = TRUNC(SYSDATE)
  ) LOOP
    PIPE ROW (transaction_rec(rec.trans_id, rec.amount, rec.trans_date));
  END LOOP;
  RETURN;
END;
/
```

SQL*Plus

Scenario: Oracle Pipelined Table Functions (2/2)

```
SELECT * FROM TABLE(get_today_transactions);

-- This returns the daily transactions as if you're querying a table. Very useful in reporting or joining with other tables.
```

Step 3: Query It Like a Table!

Why Use Pipelined Functions in Banking?

- Great for **reporting** (fetching rows for BI dashboards)
- **Efficient** for large results – returns rows **as they're processed**
- Pluggable in **views**, **joins**, **materialized views**, etc.
- Works with **PL/SQL logic** but **behaves like a table**

```
-- Join with customers
SELECT c.name, t.amount
FROM customers c
JOIN TABLE(get_today_transactions) t ON c.cust_id = t.trans_id;
```

Use in Views or Joins

Oracle Pipelined Table Functions

A **pipelined table function** returns a collection (like a table) that can be queried in SQL as if it were a table. Unlike regular functions, pipelined functions allow results to be "streamed" back row by row using the PIPE ROW syntax.

Key Components

1. **Object Type**: Defines the structure of a row.
2. **Table Type**: A collection type based on the object type.
3. **Pipelined Function**: Uses PIPELINED keyword and PIPE ROW statements.

Benefits

- **Streaming Results**: Rows are returned one at a time.
- **Parallel Execution**: Can be combined with PARALLEL_ENABLE for performance.
- **SQL-Friendly**: You can use it in SELECT, JOIN, WHERE clauses.

Create an Object Type

```
-- Step 1: Create an object type
CREATE OR REPLACE TYPE emp_obj AS OBJECT (
  emp_id NUMBER,
  emp_name VARCHAR2(100)
);
```

Create an Object Type

```
-- Step 2: Create a table type of the object
CREATE OR REPLACE TYPE emp_tab AS TABLE OF emp_obj;
```

Pipeline Function

```
-- Step 3: Create a pipelined function
CREATE OR REPLACE FUNCTION get_employees RETURN emp_tab PIPELINED IS
BEGIN
  PIPE ROW(emp_obj(1, 'Alice'));
  PIPE ROW(emp_obj(2, 'Bob'));
  RETURN;
END;
/
```

Output

```
-- Step 4: Use it in SQL
SELECT * FROM TABLE(get_employees);
```

Script Output | Query Result | SQL | All Rows Fetched: 2 in 0.006 seconds

	EMP_ID	EMP_NAME
1	1	Alice
2	2	Bob

```
-- Step 4: Use it in SQL
SELECT * FROM TABLE(get_employees);
```

Oracle Pipelined Table Functions: When and Why?

When to Use Pipelined Table Functions

Scenario	Description
1. Processing Large Datasets	When working with large volumes of data that are expensive to load all at once. Pipelined functions process and return rows one at a time—ideal for scalability.
2. Embedding Complex PL/SQL Logic in SQL Queries	Useful when you want to encapsulate business logic (loops, conditions, calculations) in PL/SQL and expose it directly to SQL for querying.
3. Streaming Data	Enables on-the-fly data transformation and delivery without needing to wait for the full dataset to be processed. Great for real-time pipelines or live reporting.

Drawbacks of Pipelined Table Functions

Limitation	Description
1. Increased Complexity	Requires defining object types, collection types, and writing procedural code. More complex than simple SQL queries or regular functions.
2. Limited Benefits for Small Datasets	For small datasets, the performance improvement is often negligible, making the added complexity unnecessary.
3. Debugging and Maintenance	Harder to debug, trace, and maintain due to procedural flow and type dependencies.
4. Overhead with Context Switching	Frequent switches between SQL and PL/SQL engines may add overhead if not designed efficiently.

Summary

Use Them When…	Avoid If…
You need to **stream large results**	You're working with **small/simple datasets**
You want **custom row-by-row logic**	Simpler logic can be done in pure SQL
SQL-only solutions are **too limiting**	You want **minimal overhead and maintenance**

Scenario: Benefits of Oracle Pipelined Table Functions in ETL

Scenario
You want to retrieve **allowances** for a given employee using a **pipelined table function**. The function returns a **table** with the following fields:

- ALLOWANCE_ID
- ALLOWANCE_AMOUNT
- EFFECTIVE_DATE

Step 1: Define an Object Type
Define a custom object to represent the structure of an individual allowance record.

```
-- Create an object type to represent one allowance record
CREATE OR REPLACE TYPE allowance_record AS OBJECT (
    ALLOWANCE_ID       NUMBER,
    ALLOWANCE_AMOUNT   NUMBER,
    EFFECTIVE_DATE     DATE
);
/
```

Step 3: Create the Pipelined Table Function
This function returns a table of allowances for a given employee using a pipeline approach.

```
-- Create a pipelined function to retrieve employee allowances
CREATE OR REPLACE FUNCTION get_employee_allowances(emp_id IN NUMBER)
RETURN allowance_table PIPELINED
AS
BEGIN
    -- Loop through each matching record from the employee_allowances table
    FOR rec IN (
        SELECT allowance_id, allowance_amount, effective_date
        FROM employee_allowances
        WHERE employee_id = emp_id
    )
    LOOP
        -- Pipe each record into the output stream
        PIPE ROW(allowance_record(rec.allowance_id, rec.allowance_amount,
rec.effective_date));
    END LOOP;

    -- Indicate completion of the function
    RETURN;
END;
/
```

Scenario: Oracle Regular & Pipelined Table Functions – Example I

You want to retrieve **allowances** for a given employee using a **pipelined table function**. The function returns a **table** with the following fields:

- ALLOWANCE_ID
- ALLOWANCE_AMOUNT
- EFFECTIVE_DATE

Step 1: Define an Object Type

```
-- Create an object type to represent one allowance record
CREATE OR REPLACE TYPE allowance_record AS OBJECT (
    ALLOWANCE_ID       NUMBER,
    ALLOWANCE_AMOUNT   NUMBER,
    EFFECTIVE_DATE     DATE
);
/
```

Step 2: Define a Table Type

```
-- Create a table type (collection) of allowance_record objects
CREATE OR REPLACE TYPE allowance_table AS TABLE OF allowance_record;
/
```

Step 3: Create the Pipelined Table Function

```
-- Create a pipelined function to retrieve employee allowances
CREATE OR REPLACE FUNCTION get_employee_allowances(emp_id IN NUMBER)
RETURN allowance_table PIPELINED
AS
BEGIN
    -- Loop through each matching record from the employee_allowances table
    FOR rec IN (
        SELECT allowance_id, allowance_amount, effective_date
        FROM employee_allowances
        WHERE employee_id = emp_id
    )
    LOOP
        -- Pipe each record into the output stream
        PIPE ROW(allowance_record(rec.allowance_id, rec.allowance_amount,
rec.effective_date));
    END LOOP;

    -- Indicate completion of the function
    RETURN;
END;
/
```

Step 4: Call the pipeline function

```
SELECT * FROM TABLE(get_employee_allowances(101));
```

Oracle Regular & Pipelined Table Functions – Example II

Step 1: Define an Object Type

```
-- Create an object type to represent one allowance record
CREATE OR REPLACE TYPE allowance_record AS OBJECT (
    ALLOWANCE_ID       NUMBER,
    ALLOWANCE_AMOUNT   NUMBER,
    EFFECTIVE_DATE     DATE
);
/
```

Step 2: Define a Table Type

```
-- Create a table type (collection) of allowance_record objects
CREATE OR REPLACE TYPE allowance_table AS TABLE OF allowance_record;
/
```

Output

```
SELECT *
FROM TABLE(get_employee_allowances(50));  -- Replace any valid employeeID
```

Script Output | Query Result | Query Result 1 | Query Result 2 | Query Result 3 | Query Result 4

SQL | All Rows Fetched: 3 in 0.004 seconds

	ALLOWANCE_ID	ALLOWANCE_AMOUNT	EFFECTIVE_DATE
1	21	2555000	10-NOV-24
2	24	6000	01-NOV-24
3	3	3000	29-SEP-24

Step 3: Create the Pipelined Table Function

```
-- Create a pipelined function to retrieve employee allowances
CREATE OR REPLACE FUNCTION get_employee_allowances (
    p_employee_id NUMBER
) RETURN allowance_table PIPELINED IS

BEGIN
    -- Loop through all applicable allowances for the given employee
    FOR r IN (
                SELECT a.ALLOWANCE_ID, a.ALLOWANCE_AMOUNT, a.EFFECTIVE_DATE
                FROM ALLOWANCES a
                JOIN EMPLOYEES e ON a.ROLE_ID = e.ROLE_ID
                WHERE e.EMPLOYEE_ID = p_employee_id
          AND a.IS_APPLICABLE = 'Y'
          AND a.EFFECTIVE_DATE <= SYSDATE
    ) LOOP
        -- Pipe each selected row into the result set
                PIPE ROW(allowance_record(r.ALLOWANCE_ID, r.ALLOWANCE_AMOUNT,
          r.EFFECTIVE_DATE));
    END LOOP;

    RETURN;  -- End of the function; return the pipelined data
END get_employee_allowances;
/
```

Step 4: Call the pipeline function

```
SELECT *
FROM TABLE(get_employee_allowances(101));  -- Replace 101 with any valid
EMPLOYEE_ID
```

Question: Adjusting Employee Salaries Using a PL/SQL Package Procedure

Question:

You are provided with a package named employee_management_pkg which contains:

1. A **function** calculate_bonus(salary IN NUMBER) RETURN NUMBER that returns 10% of the given salary as a bonus.
2. A **procedure** adjust_employee_salary(emp_id IN NUMBER, salary IN OUT NUMBER, new_salary OUT NUMBER) that:
   - Retrieves the employee's current salary from the EMPLOYEES table based on the provided emp_id.
   - Calculates a bonus using the calculate_bonus function **if the salary is below 50,000** and increases the salary by the bonus.
   - Otherwise, increases the salary by a fixed amount of **5,000**.
   - Updates the new salary in the EMPLOYEES table.
   - Handles exceptions if the employee ID is not found or if another error occurs.

Task:

Write an **anonymous PL/SQL block** that:

1. Accepts an EMPLOYEE_ID as input (choose a valid one from your data),
2. Calls the procedure adjust_employee_salary from the employee_management_pkg package,
3. Displays the original and updated salary using DBMS_OUTPUT.PUT_LINE.

Bonus:

Modify your PL/SQL block to:

- Display a custom message when the salary was adjusted based on bonus vs. fixed increment.
- Include exception handling in your test block.

Solution: Adjusting Employee Salaries Using a PL/SQL Package Procedure

Package Specification

```
-- Create the package specification
CREATE OR REPLACE PACKAGE employee_management_pkg AS
    FUNCTION calculate_bonus(salary IN NUMBER) RETURN NUMBER;  -- Function parameter
    PROCEDURE adjust_employee_salary(emp_id IN NUMBER, salary IN OUT NUMBER, new_salary OUT NUMBER); -- Procedure parameters
END employee_management_pkg;
/
```

Define the package

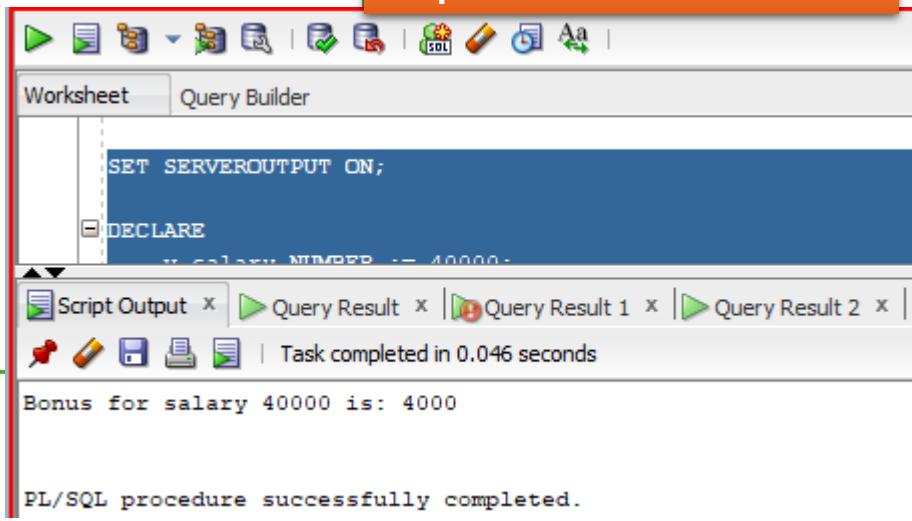
```
-- Create the package body
CREATE OR REPLACE PACKAGE BODY employee_management_pkg AS
    -- Function to calculate bonus (10% of salary)
    FUNCTION calculate_bonus(salary IN NUMBER) RETURN NUMBER IS
    BEGIN
        RETURN salary * 0.1; -- 10% bonus
    END calculate_bonus;

    -- Procedure to adjust the employee's salary and bonus
    PROCEDURE adjust_employee_salary(emp_id IN NUMBER, salary IN OUT NUMBER, new_salary OUT NUMBER) IS
        v_bonus NUMBER; -- Variable to hold calculated bonus
    BEGIN
        -- Check if the employee exists and fetch the current salary
        SELECT SALARY INTO salary
        FROM EMPLOYEES
        WHERE EMPLOYEE_ID = emp_id;

        -- Logic for adjusting salary
        IF salary < 50000 THEN
            v_bonus := calculate_bonus(salary); -- Calculate bonus
            salary := salary + v_bonus; -- Increase salary by bonus
        ELSE
            salary := salary + 5000; -- Fixed increment for high earners
        END IF;

        new_salary := salary; -- Assign updated salary

        -- Update the employee's salary in the database
        UPDATE EMPLOYEES SET SALARY = salary WHERE EMPLOYEE_ID = emp_id;

        DBMS_OUTPUT.PUT_LINE('Updated salary for employee ID ' || emp_id || ' is ' || new_salary);

    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Employee with ID ' || emp_id || ' not found.');
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
    END adjust_employee_salary;
END employee_management_pkg;
/
```

Test: Adjusting Employee Salaries Using a PL/SQL Package Procedure

Test our Function inside the package

```
SET SERVEROUTPUT ON;

DECLARE
    v_salary NUMBER := 40000;
    v_bonus NUMBER;
BEGIN
    -- Call the function from the package
    v_bonus := employee_management_pkg.calculate_bonus(v_salary);
    DBMS_OUTPUT.PUT_LINE('Bonus for salary ' || v_salary || ' is: ' || v_bonus);
END;
/
```

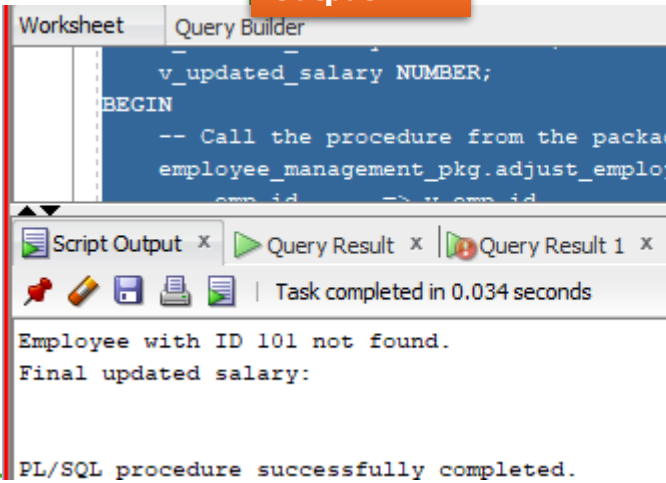
Output



Test our Procedure inside the package

```
DECLARE
    v_emp_id NUMBER := 101;          -- Replace with a valid EMPLOYEE_ID in your table
    v_current_salary NUMBER := 0;
    v_updated_salary NUMBER;
BEGIN
    -- Call the procedure from the package
    employee_management_pkg.adjust_employee_salary(
        emp_id      => v_emp_id,
        salary      => v_current_salary,
        new_salary  => v_updated_salary
    );

    -- Display the results
    DBMS_OUTPUT.PUT_LINE('Final updated salary: ' || v_updated_salary);
END;
/
```

Output



Creating a Pipelined Table Function for Real-Time Salary Adjustment Insights in HR Systems (1/2)

Scenario-Based Question:
You are working as a PL/SQL developer for an HR management system in a large company. The Human Resources department often needs to review salary adjustments and bonus eligibility for employees without making direct changes to the database. To improve performance and enable flexible reporting, the team has requested a solution that can stream data in a queryable format.

The company uses the following simplified EMPLOYEES table:

```
EMPLOYEE_ID   NUMBER
FIRST_NAME    VARCHAR2(50)
LAST_NAME     VARCHAR2(50)
ROLE_ID       NUMBER
SALARY        NUMBER
```

The bonus policy is as follows:

- If an employee's salary is **less than 50,000**, they receive a **bonus of 10% of their current salary**.
- Otherwise, they receive a **fixed bonus of 5,000**.
- You are required to develop a **pipelined table function** that:
- Loops through all employees in the EMPLOYEES table.
- Calculates their bonus using the logic described above.
- Returns the following details for each employee:
    - EMPLOYEE_ID
    - SALARY (original)
    - BONUS (calculated)
    - NEW_SALARY (SALARY + BONUS)

The function should **not update the table**, but provide a result set suitable for analysis.

Creating a Pipelined Table Function for Real-Time Salary Adjustment Insights in HR Systems (2/2)

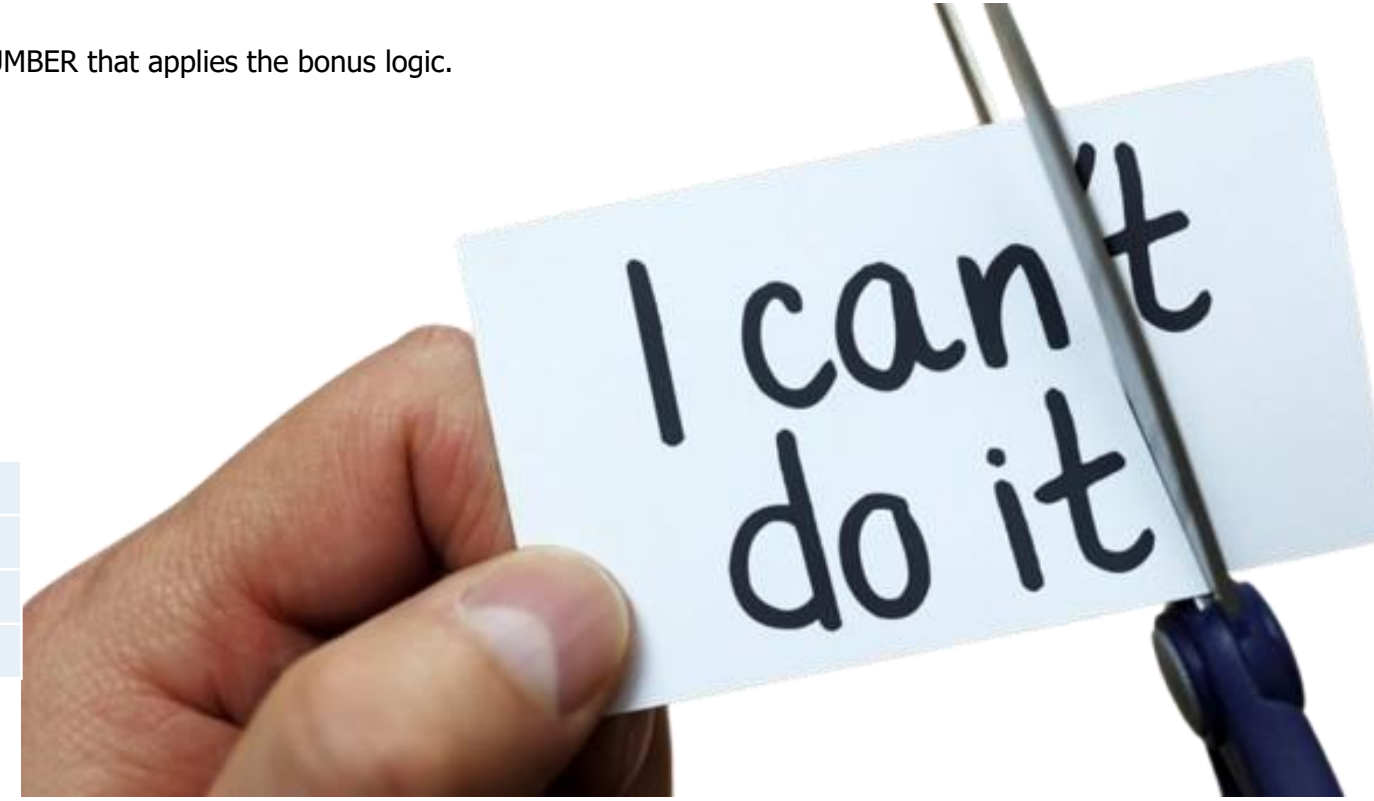
Task Instructions:

1. **Create an object type** (e.g., salary_bonus_rec) to represent a record with the required fields:
   EMPLOYEE_ID, SALARY, BONUS, and NEW_SALARY.
2. **Create a collection type** (e.g., salary_bonus_tab) as a table of the object type.
3. **Create a helper function** (optional) calculate_bonus(salary NUMBER) RETURN NUMBER that applies the bonus logic.
4. **Create the pipelined function** (e.g., get_adjusted_salaries) that:
   - Returns salary_bonus_tab PIPELINED
   - Pipes each employee's computed values
5. Finally, test the function by querying it using:

SELECT * FROM TABLE(get_adjusted_salaries);

The output should look like:

EMPLOYEE_ID	SALARY	BONUS	NEW_SALARY
101	40000	4000	44000
102	65000	5000	70000
…	…	…	…



Question: Oracle Pipeline

Question:
Create a pipelined function that returns information about products and their pricing. Your task is to write a function named get_product_pricing that accepts a product category ID as input and returns a pipelined table containing the product ID, product name, and product price for each product in that category.

Steps:
1. Create a table named products with the following columns:
    - product_id (NUMBER): Primary key, representing the product ID.
    - product_name (VARCHAR2(100)): Name of the product.
    - category_id (NUMBER): Represents the category ID of the product.
    - price (NUMBER): Price of the product.
2. Insert sample data into the products table for at least three categories, each with several products.
3. Define an object type named product_pricing_record with the following attributes:
    - product_id (NUMBER)
    - product_name (VARCHAR2(100))
    - price (NUMBER)
4. Create a table type named product_pricing_table based on product_pricing_record to store multiple rows of product_pricing_record.
5. Write the get_product_pricing function to:
    - Accept a category_id as an input parameter.
    - Return a pipelined table of type product_pricing_table.
    - Loop through each product in the specified category and return each row in the pipeline with the product's ID, name, and price.

Collaboration is essential for effective learning! Work as a team, share your strengths, ask questions, and support each other. Focus on understanding the concepts together—not just finishing the tasks. Enjoy the journey and learn from one another!

Conclusion

Key Takeaways

Oracle Packages:

- **Modularity**: Packages group related procedures, functions, variables, and cursors into a single unit for better organization.
- **Encapsulation**: Public specifications expose only what's necessary, hiding implementation details.
- **Reusability & Maintainability**: Once compiled, packages can be reused across different PL/SQL programs, simplifying maintenance.
- **Performance**: Packages load into memory once and allow faster access to bundled objects.

Pipelined Table Functions:

- **Set-Based Processing**: Allow functions to return data row-by-row (streaming), improving memory efficiency.
- **Queryable Results**: Enable using SELECT * FROM TABLE(function_name(...)), integrating seamlessly with SQL queries.
- **Ideal for ETL**: Efficiently transform and stream large datasets without needing to materialize intermediate results.
- **Custom Output Structures**: Use object and collection types to return complex or structured data on the fly.

Pipeline Use in ETL:

- **Extract**: Fetch relevant data from tables.
- **Transform**: Apply business rules (e.g., bonus logic, salary adjustments).
- **Load**: Results can be inserted into staging/reporting tables or viewed dynamically.

Conclusion: Combining Oracle Packages and Pipelined Table Functions is a powerful way to build performant, modular, and scalable PL/SQL-based data transformation pipelines — essential for real-time analytics and ETL workflows.

Keep Learning!